
Part III

Advanced Strategies

Astrophysical Recipes (Second Edition)

The art of AMUSE

Simon Portegies Zwart, Steve McMillan and Steven Rieder

Chapter 8

Elementary Coupling Strategies

I would rather have questions that can't be answered than answers that can't be questioned.

—R. Feynman

The simplest multiscale or multiphysics coupling strategy is the linear propagation of information from one code to another. In some sense, one code produces the conditions that another code uses to continue the calculation. These couplings are often unidirectional; one code informs another, but there is no back propagation of information. So long as the response from one code to the other is linear, we can refrain from complex code-coupling strategies, as described in the next chapter (Chapter 9). Linear coupling strategies lead to new AMUSE applications without increasing your script's complexity, while increasing its applicability.

8.1 Multicode Coupling Problems

In this chapter, we discuss how one code can produce self-consistent input or boundary conditions for another code without using some intermediate state, such as a file. We encountered such coupled problems in Section 4.5.1, where a gravitational dynamical simulation required interrupts in order to resolve collisions, in Section 5.2.10 for treating stellar explosions, and in Section 6.3.5 where we discussed how a hydrodynamics solver can be stopped to allow a special treatment of high-density clumps to be addressed nonhydrodynamically.

In these examples the physical solvers were not coupled, but instead we adopted a simple linear information propagation strategy ($A \rightarrow B$), in which one code (A) generated the initial conditions for another (B), but there was no back coupling from the other code (B) to the first (A).

In this chapter, we focus on weakly coupled problems in which one code generates the data for another code to proceed, after which a third code (or the first one)

continues or is (re)started. Towards the end of this chapter, in Section 8.7.2, we will discuss two different strategies of multiscale and multiphysics coupling. These couplings are relatively simple solutions to the very complex but weakly coupled problem of multiscale and multiphysics simulations. The numerics become considerably more complex once a code responds nonlinearly to small perturbations induced by another code. This more advanced topic of code coupling is discussed in Chapter 9.

A word of caution is in order here. There is probably no unique way to combine numerical simulations of different physical domains. Although many researchers claim self-consistency in their numerical solutions, this rarely is actually the case. The AMUSE approach described here lends itself well to problems in which the physics is weakly coupled, but even in those cases validation is a dark art rather than a well-documented science.

8.2 Advanced Data Handling

8.2.1 Dedicated Channels and Copy Operations

Key to coupling codes is the ability to transfer data from one code to another, taking the proper unit conversions into account. In AMUSE we achieve this objective by the use of channels. Channels are one of the cornerstones of AMUSE, allowing one code to modify the data on which another code operates, as we now discuss.

Channels are used to communicate data between modules and the driving Python script. We encountered channels previously in Section 2.2.3, where we learned how to copy data from an N -body code to the AMUSE framework, and vice versa. Here, we extend that discussion. Recall that channels are dedicated MPI (or sockets) communication links that do not need to be reestablished on every use. While it is possible to access code data directly from the running module (e.g. as `module.particles.mass`), we strongly encourage the use of channels because they are faster, more efficient, more transparent, and result in cleaner code compared to directly accessing the module's particle set. Perhaps most important, they are independent of specific hardware and software implementations of interprocess communication. For monophysics solvers, channels may seem like overkill, but with multiphysics solvers they are extremely useful, if not essential.

To run stellar evolution and gravity concurrently, we require three channels: one to copy data from the gravity solver to the local Python framework, one to copy data from the stellar-evolution code to the framework, and one to copy data from the framework to the gravity code. This can be done as follows:

```
channel_from_gravity = gravity.particles.new_channel_to(bodies)
channel_from_stellar = stellar.particles.new_channel_to(bodies)
channel_to_gravity = bodies.new_channel_to(gravity.particles)
```

The local particles (called `bodies` in this example) can contain more information than either of the running codes provide. The first and second channels are used to copy data from the specific modules to the framework, whereas the third channel allows us to write to the gravitational dynamics module (see Section 2.2.3). In this

case, we do not require a channel from the framework to the stellar-evolution code because the gravitational dynamics do not affect the stellar evolution. That would change if we were to incorporate collisions, in which case the mass of a merger product, and therefore the evolution of that star, would be affected by the merger process.

Code example 8.1 presents an initialization sequence for combining stellar evolution with gravitational dynamics, demonstrating that communication through the Python script is often not necessary because codes can communicate directly with each other through channels. We can arrange for the stellar-evolution code to communicate directly with the gravity solver by declaring a direct channel between the two modules and copying the relevant information,

```
channel_from_stellar_to_gravity = stellar.particles.new_channel_to(
    gravity.particles
)
```

The data stream still passes through the AMUSE framework, to ensure proper unit conversion, but in this case the local `bodies` particle set is unaffected. This is a bit tricky because now the copies of the data in the various running codes are not synchronized with the data in the local Python script.

After the particle set is constructed, the codes initialized, and the channels created, these channels can now be used to communicate between the running modules and the framework. For example, the gravity solver affects stellar positions and velocities, whereas the stellar-evolution module affects stellar masses. After taking steps in `stellar`, the new stellar masses must be copied to the `gravity` module before the next gravity step occurs.

Code example 8.2 presents an example of the sequence of calling the codes and the associated copy commands. Here, we assume that the channels `channel_stellar_to_gravity` and `channel_gravity_to_framework` are constructed to copy data from `stellar` to `gravity` and back from `gravity` to the framework. Between the calls to `gravity.evolve_model` and `stellar.evolve_model`, there is no need to synchronize the data because the gravity module does not affect the parameters used by the stellar-evolution code.

In Code example 8.2, we adopted an interlaced time-stepping strategy, which is a slight improvement over the linear temporal interstepping discussed so far. The more elaborate event-driven approach will be discussed in Section 8.4.1, and illustrated in Code example 8.7.

At the end of the snippet shown here, the `bodies` in the script are up to date and can be used to write a snapshot for later analysis, as we do in the last line of the script, where the `bodies` array is saved to a file called `filename`. Note that the order in which channels are called is important for those parameters that share the same name but may conceivably have different meanings in different codes. For example, both codes here use the particle attribute `mass`, which means essentially the same thing in each code. However, the attribute `radius` can and does mean different things in different modules, requiring some care when managing data transfer (see, e.g., Section 8.7.1).

```

31 masses = new_salpeter_mass_distribution(number_of_stars, mass_min, mass_max)
32 mass_total_init = masses.sum()
33 converter = nbody_system.nbody_to_si(mass_total_init, radius_virial)
34 bodies = new_king_model(number_of_stars, W0, convert_nbody=converter)
35 bodies.mass = masses
36 bodies.scale_to_standard(convert_nbody=converter)
37
38 stellar = Sse()
39 stellar.parameters.metallicity = metallicity
40 stellar.particles.add_particle(bodies)
41 bodies.radius = stellar.particles.radius
42
43 gravity = Ph4(converter)
44 gravity.parameters.timestep_parameter = 0.01
45 gravity.particles.add_particles(bodies)
46
47 channel_from_stellar_to_gravity = stellar.particles.new_channel_to(
48     gravity.particles
49 )
50 channel_from_gravity_to_framework = gravity.particles.new_channel_to(bodies)
51

```

Code example 8.1: Initializing a simple solver combining stellar evolution with gravity.

```

54 energy_total_init = gravity.kinetic_energy + gravity.potential_energy
55 energy_difference_gr = 0 | energy_total_init.unit
56
57 time = 0.0 | time_end.unit
58 while time < time_end:
59     time_step = min(stellar.particles.time_step.amin(), time_end - time)
60
61     stellar.evolve_model(time + time_step / 2)
62     channel_from_stellar_to_gravity.copy()
63
64     energy_total_gr = gravity.kinetic_energy + gravity.potential_energy
65     gravity.evolve_model(time + time_step)
66     energy_difference_gr += (
67         gravity.kinetic_energy + gravity.potential_energy - energy_total_gr
68     )
69
70     stellar.evolve_model(time + time_step)
71     channel_from_stellar_to_gravity.copy()
72     channel_from_gravity_to_framework.copy()
73
74     time += time_step
75     write_set_to_file(bodies, filename, append_to_file=True)
76

```

Code example 8.2: Event loop combining stellar evolution with gravity.

8.2.1.1 Nonsymmetric Cross Synchronization of Specific Attributes

A certain number of attribute names have specific purposes for different modules in AMUSE. The radius, used in particle sets, for example, is used in an N -body code as the collision radius that triggers the associated stopping conditions (see Section

```

70 masses = new_salpeter_mass_distribution(N, Mmin, Mmax)
71 Mtot_init = masses.sum()
72 converter=nbody_system.nbody_to_si(Mtot_init,Rvir)
73 bodies = new_king_model(N, W0,convert_nbody=converter)
74 bodies.mass = masses
75 bodies.scale_to_standard(convert_nbody=converter)
76
77 gravity = ph4(converter)
78 gravity.parameters.timestep_parameter = 0.01
79 gravity.particles.add_particles(bodies)
80 collision_detection = gravity.stopping_conditions.collision_detection
81 collision_detection.enable()
82
83 stellar = SeBa()
84 stellar.parameters.metallicity = z
85 stellar.particles.add_particle(bodies)
86 stellar.evolve_model(0|units.Myr)
87
88 supernova_detection = stellar.stopping_conditions.supernova_detection
89 supernova_detection.enable()
90
91 channel_from_se = stellar.particles.new_channel_to(bodies)
92 channel_from_gd = gravity.particles.new_channel_to(bodies)
93 channel_to_gd = bodies.new_channel_to(gravity.particles)
94 channel_from_se.copy_attributes(["mass", "radius", "age",
95                                "temperature", "luminosity"])

```

Code example 8.7: Declaration of the solvers and data structures used in Code example 8.8.

4.5.1). In the stellar-evolution modules, the same attribute refers to the photospheric radius of the star. This is confusing, and at times may even cause the script to behave inappropriately.

To prevent such name confusion, we would like to be able to copy a specific attribute from a running code to another local attribute name in the script. This can be realized through channels, in case the receiver's name does not match that of the sender.

The physics modules tend to use generic terms such as mass, radius, position, etc. In a complicated production script, one may wish to copy the mass from the particle set of a stellar-evolution code, `stellar.particles`, to a parameter with another name. This might happen, for example, when the gravity solver treats the combined mass of a star and a protoplanetary disk as a single particle, whereas the stellar-evolution code uses only the stellar mass to calculate the evolution of the star. In the local particle-set `bodies`, the attributes might then be called `bodies.mass_of_star_and_disk` for the total mass of the star and the disk, and `bodies.radius_of_star_and_disk` for the equivalent radius, whereas the mass of the star alone could be called `bodies.stellar_mass`. In the main script we declare the parameter `bodies.mass_star_and_disk=bodies.stellar_mass+bodies.disk_mass`.

We can tune the various channels to copy the right parameters. In this case we can, for example, say

```

channel_to_stellar = bodies.new_channel_to(
    stellar.particles,
    attributes=["radius"],
    target_names=["radius"]
)
channel_to_gravity = bodies.new_channel_to(
    gravity.particles,
    attributes=["mass_of_star_and_disk",
               "radius_of_star_and_disk"],
    target_names=["mass", "radius"]
)

```

When the copy function is called, only the specific named attributes are copied. Similar channels can be declared for the reverse operations, such as copying the mass from the stellar-evolution module back to the local particle set

```

channel_from_stellar = stellar.particles.new_channel_to(
    bodies,
    attributes=["mass"],
    target_names=["stellar_mass"]
)

```

In the main event loop we would write

```

channel_to_stellar.copy()
channel_to_gravity.copy()

```

to copy just the masses from the local particle set into the stellar-evolution code, and the masses and radii from the local set into the gravity code. Similarly, we would use

```

channel_from_stellar.copy()

```

after the stellar-evolution step to copy the updated stellar masses back into local memory. We present an example of how this works in practice in Section 8.4.4.

8.2.2 Particle Subsets and Supersets

Sometimes, it is desirable to perform certain tasks on a subset of the particles in a set. This can be accomplished by generating new particle sets, but managing multiple related particle sets can unnecessarily complicate the bookkeeping, increase the chance of mistakes, and reduce readability. We can achieve the same result using channels.

We demonstrate this approach by simulating a system of stars with planetesimal disks around them. All particles should be evolved using the gravitational dynamics module, but only the stars should be affected by stellar evolution. To do this, we define two particle sets, called `planetesimals` and `stars`. Both sets contain masses, positions, and velocities, but the `stars` set also includes stellar data, such as radius, luminosity, and temperature.

We send `stars` to the stellar-evolution code:

```
stellar.particles.add_particle(stars)
```

after which we can call `evolve_model` to evolve them. But, we send both `stars` and `planetesimals` to the gravity module:

```
gravity.particles.add_particle(stars)
gravity.particles.add_particle(planetesimals)
```

We subsequently declare the appropriate channels to copy data from the physics modules to the framework:

```
channel_from_stellar_to_stars = stellar.particles.new_channel_to(stars)
channel_from_gravity_to_stars = gravity.particles.new_channel_to(stars)
channel_from_gravity_to_planetesimals \
    = gravity.particles.new_channel_to(planetesimals)
```

and a similar set of channels from the framework to the physics modules.

In many cases, however, it may be easier to declare a *superset* of particles

```
gravitating_bodies = ParticlesSuperset([planetesimals, stars])
```

After this, we can treat `gravitating_bodies` as the particles affected by gravity, keeping the `planetesimals` separate from the `stars` for the stellar evolution. Sending the superset to the gravity module can be accomplished with

```
gravity.particles.add_particles(gravitating_bodies)
```

Only the stars are sent to the stellar-evolution module.

In order to guarantee consistency among the various parameters in the particle subsets, we create channels to and from the gravity module, and one channel from the stellar-evolution code:

```
channel_to_gravity = gravitating_bodies.new_channel_to(gravity.particles)
channel_from_gravity = gravity.particles.new_channel_to(gravitating_bodies)
channel_from_stellar = stellar.particles.new_channel_to(stars)
```

Now, we can easily copy particle attributes after the gravity or stellar-evolution steps, e.g.,

```
channel_from_stellar.copy_attributes(["mass", "radius"])
```

We can select subset of particles within a particle set by using standard Python tools. For example, suppose we have a superset of stars and planetesimals, as in `gravitating_bodies` just discussed. Filtering out the planetesimals can be realized by selecting on their masses—since the planetesimals have low masses, we can write

```
mass_min = 1 | units.MEarth
low_mass = gravitating_bodies[gravitating_bodies.mass < mass_min]
```

to select objects with masses less than 1 Earth mass. In principle, we could use the more arcane but versatile `Pythonlambda` function. Regrettably, however, as of 27 November 2023 Lambda functions are no longer supported in Python in tandem with the AMUSE `select` operation.

8.2.3 Interrupting Codes and Stopping Conditions

Sometimes it is necessary to have a community code stop and hand control back to the framework, leaving subsequent decisions to the user. As discussed in Section 4.5.1, Section 5.2.10, and Section 6.3.5 (interestingly enough the radiative-transfer module has no stopping conditions) we call these events *stopping conditions*. In Section A.5.2, we present a more complete overview of the various stopping conditions. There are many reasons why a community code should stop and return control to the user script, and the circumstances may be quite different for each physical domain. Stopping conditions are intrusive to the community code, and therefore are inconsistent with the AMUSE philosophy of “keep community codes pristine.” Nevertheless, we have taken the step of implementing stopping conditions in most AMUSE community codes. The details vary from one code to another. Barnes–Hut tree codes, for example, are generally not built to detect physical collisions between stars, while in regularized N -body codes, a “collision” may have an entirely different meaning. In all cases, checks for physical (or other) collisions can usually be incorporated into the interface code or at a high level in the community code without entailing sweeping changes in the source.

In an AMUSE script, one can initiate stopping conditions after initializing the worker code. In a gravity module this is realized by

```
detect_collision = gravity.stopping_conditions.collision_detection
detect_collision.enable()
```

Many, but not all, AMUSE N -body codes have collision detection enabled. Those codes that do not will produce a warning message when the user code attempts to set a stopping condition, but subsequently they simply continue without the ability to stop, offering no other indication that stopping is disabled.

In the time-evolution loop, you can check for a stopping condition on return from `evolve_model()` and take appropriate action:

```
if detect_collision.is_set():
    for cp in range(len(detect_collision.particles(0))):
        particles_in_code = Particles(
            particles=[detect_collision.particles(0)[cp],
                       detect_collision.particles(1)[cp]]
            local_colliding_particles = \
                particles_in_code.get_intersecting_subset_in(bodies)
            merge_two_particles(gravity, local_colliding_particles)
            gravity.particles.synchronize_to(bodies)
```

The N -body integrator checks for the exception after each internal step and returns to the calling function when the event (in this case, a collision) is detected. The N -body system is synchronized before returning to the framework to allow the

calling function to perform further tasks on the datasets without corrupting the integrity of the N -body data.

The flag `detect_collision.is_set()` returns True if the stopping condition is met, after which the code for resolving stopping conditions is invoked. Inside this code, the bodies that caused the community code to stop are returned to the Python framework and the user script must deal with the exception. In this example, the gravity solver stops when two bodies approach within a distance smaller than the sum of their radii. The resolution of the exception in this case is to delete the colliding particles and insert a new particle at their center of mass. The following routine accomplishes this as follows:

```
def merge_two_particles(gravity, particles_in_encounter):
    new_particle = Particles(1)
    new_particle.mass = particles_in_encounter.total_mass()
    new_particle.position = particles_in_encounter.center_of_mass()
    new_particle.velocity = particles_in_encounter.center_of_mass_velocity()
    new_particle.radius = particles_in_encounter.radius.sum()
    gravity.particles.add_particles(new_particle)
    gravity.particles.remove_particles(particles_in_encounter)
```

In this routine, a new particle is created and the center-of-mass data of the merging particles are copied to the new particle and added to the `gravity` instantiation. The two merging stars are not needed anymore, so they are removed from `gravity`. Best practice probably requires that we also flag the event, writing the merging particles to a file before removing them from the system (not shown here). Removal is achieved by explicitly removing the particles from the local `bodies`, adding the center of mass, and synchronizing `gravity.particles` with the local data:

```
gravity.particles.synchronize_to(bodies)
```

Of course, for all this to happen, one must set the radii of the particles in advance. For example, you could write

```
bodies.radius = 0.001 | nbody_system.length
```

just after the particles have been initialized, but before they are sent to the community module. When working with physical units, that line might look as follows

```
bodies.radius = 10 | units.RSun
```

8.2.4 Collision Handling

We decide when the point-mass approximation fails based on the sizes of two stars (as determined by the stellar-evolution code) and the distance between them (from the N -body code). A collisional stopping condition is triggered when the distance between two stars becomes smaller than the sum of their radii.

Because a stopping condition is an exception, it must be checked for and addressed as soon as possible after it is raised (see Section A.5.2). In the case of a collision, once the gravity module reports the exception, we should next evolve the stars to the moment the collision was detected. It is possible (although not common) that the collision may be avoided by a drastic change in stellar radius due to stellar evolution. For example, the gravity module may decide that two giant stars have collided, but when stellar evolution is applied these stars may turn into (non-colliding) compact objects before the collision is resolved. In such a case, the collision should probably not occur, and the dynamical integration can simply continue.

Synchronization between the modules (here gravity and stellar evolution) is very important. Recall that there are usually two copies of the data for each module—one in local framework (Python) memory, the other in the module itself. The module data is up to date at the end of a step, and we must synchronize the modules with the local framework before resolving the stopping condition. To manage a collision between two stars, the framework needs to know the stellar ages, masses, radii, luminosities, and temperatures. Some of this information comes from the gravity module, the rest from the stellar-evolution module.

Note that we must keep track of two different “radii”—the natural stellar radius in the stellar-evolution module, and the collision radius used by the gravity code to identify close dynamical encounters (see Section 8.7.1). Probably the safest way to manage this is by initializing two separate local variables in the framework particle set. Selective channels can be declared to ensure proper communication between the modules and the framework script:

```
channel_from_stellar = stellar.particles.new_channel_to(bodies,
    attributes=["mass", "radius", "age"],
    target_names=["mass", "radius", "age"])
channel_from_gravity = gravity.particles.new_channel_to(bodies,
    attributes=["x", "y", "z", "vx", "vy", "vz", "mass", "radius"],
    target_names=["x", "y", "z", "vx", "vy", "vz", "mass", "collision_radius"])
channel_to_gravity = bodies.new_channel_to(gravity.particles,
    attributes=["mass", "collision_radius"], target_names=["mass", "radius"])
```

In `channel_from_gravity`, we explicitly copy all the positions and velocities to ensure that the particle set in the framework is consistent with the gravity module. Later in the event loop (see Code example 8.3) we will simply call the copy operator for the various channels. We copy data from the stellar evolution to the framework with

```
channel_from_stellar.copy()
```

and copy data from the N -body code to the framework with

```
channel_from_gravity.copy()
```

These lines ensure that the masses and radii used in the gravitational module are consistent with those in the stellar-evolution module and in the framework.

```

82     while time < t_end:
83         time += dt
84
85         bodies.radius *= 1.e+5
86         channel_from_framework_to_gd.copy_attributes(["mass", "radius"])
87         E_dyn = gravity.kinetic_energy + gravity.potential_energy
88         gravity.evolve_model(time)
89         dE_dyn = E_dyn - (gravity.kinetic_energy + gravity.potential_energy)
90
91         Nenc += resolve_stellar_encounter(time, stellar, gravity, bodies,
92                                           stopping_condition)
93
94         E_stellar = gravity.kinetic_energy + gravity.potential_energy
95         stellar.evolve_model(time)
96         dE_stellar = E_stellar - (gravity.kinetic_energy + gravity.potential_energy)
97
98         channel_from_gd_to_framework.copy()
99         channel_from_se_to_framework.copy_attributes(["age", "mass", "radius"])
100
101         write_set_to_file(bodies.savepoint(time), filename, 'hdf5', append_to_file=
102                           True)
103         print_diagnostics(time, bodies.mass.sum(), E_dyn, dE_dyn, dE_coll,
104                           dE_stellar)

```

Code example 8.3: Main code declaration and event loop for simulating a star cluster with collisions.

Stars are small objects, and collide infrequently. In order to make more stars collide (for debugging or illustrative purposes), we can increase the stellar radii used by the gravitational dynamics, multiplying the radii provided by the stellar-evolution code by some factor. This has to be done after the stellar-evolution information is copied back into the local particle set. The magnified particle-set data must then be copied into the gravity module's particle set to ensure that the dynamics is aware of the new stellar radii:

```

stellar.evolve_model(time)
channel_from_stellar.copy()
bodies.collision_radius = 1.0e5 * bodies.radius
channel_to_gravity.copy()
gravity.evolve_model(time)

```

This is a pretty dirty trick, but it will guarantee many interesting collisions!

Code example 8.3 presents part of the main routine for simulating the dynamical evolution of a cluster of stars, including stopping conditions for collisions. The function that resolves the collisions is presented in Code example 8.4. The same routine is also used in Section 8.5.1, where we discuss an event-driven simulation of a star cluster with collisions.

How do we model the merger product? If we are using a parameterized stellar-evolution code, the structure of the merger product cannot easily be determined. We can mimic the mixing of fresh hydrogen into the merger product's core by reducing the stellar age in some mass-dependent way, resulting in an effective rejuvenation of the star (see Section 5.3.12). Afterwards, we place the merged star on a new

```

28 def resolve_stellar_encounter(time, stellar, gravity, bodies, stopping_condition):
29     Nenc = 0
30     if stopping_condition.is_set():
31         E_coll = gravity.kinetic_energy + gravity.potential_energy
32         print("At time=", gravity.model_time.in_(units.Myr), "number of encounters="
33             , len(stopping_condition.particles(0)))
34         for ci in range(len(stopping_condition.particles(0))):
35             particles_in_encounter = Particles(particles=[stopping_condition.
36                 particles(0)[ci], stopping_condition.particles(1)[ci]])
37             particles_in_encounter = particles_in_encounter.
38                 get_intersecting_subset_in(bodies)
39
40             merge_two_stars(bodies, particles_in_encounter)
41             bodies.synchronize_to(gravity.particles)
42             bodies.synchronize_to(stellar.particles)
43             Nenc+=1
44             print("Resolve encounter Number:", Nenc)
45             gravity.evolve_model(time)
46             dE_coll = E_coll - (gravity.kinetic_energy + gravity.potential_energy)
47     return Nenc

```

Code example 8.4: Simple function to resolve stellar collisions.

```

13 def merge_two_stars(bodies, particles_in_encounter):
14     com_pos = particles_in_encounter.center_of_mass()
15     com_vel = particles_in_encounter.center_of_mass_velocity()
16     new_particle=Particles(1)
17     new_particle.mass = particles_in_encounter.total_mass()
18     new_particle.age = min(particles_in_encounter.age) * max(particles_in_encounter.
19         mass)/new_particle.mass
20     new_particle.position = com_pos
21     new_particle.velocity = com_vel
22     new_particle.radius = 0 | units.RSun
23     bodies.add_particles(new_particle)
24     bodies.remove_particles(particles_in_encounter)

```

Code example 8.5: Simple function to merge and rejuvenate two stars while conserving linear momentum.

evolutionary track at the new age and continue the calculation. This can be realized simply by adding a few lines to the linear momentum- (and mass-)conserving routine `merge_two_particles()` discussed in Section A.5.2, which then becomes Code example 8.5. In the next subsection, we discuss a more sophisticated approach to the problem.

8.3 Runtime Analysis and Task Offloading

AMUSE contains many analysis tools designed to operate on simulation data. Here, we describe two modules, `hop` and `Kepler`, that are commonly used in tandem with running simulation codes. There are other codes that are often associated with AMUSE, and their number is growing continuously, but this book can only present a snapshot in time of the development.

8.3.1 The Hop Package

Hop (Eisenstein & Hut 1998) is a module designed to identify and quantify structure in a distribution of points.¹ Although it was originally developed to identify clumps in a three-dimensional spatial distribution of objects (stars in a cluster or galaxies in the Universe), we can easily apply hop to any set of particle data. For example, to distinguish planetesimals in a protoplanetary disk from other debris, we might look for clumps in the space of orbital velocity, inclination, and distance from the central star. Figure 8.1 shows a small example of how hop can be used to characterize clumpy structure in a distribution of stars.

The code snippet in Code example 8.6 gives the basic calling sequence to run hop. Several free parameters guide hop in finding structure and substructure; they include

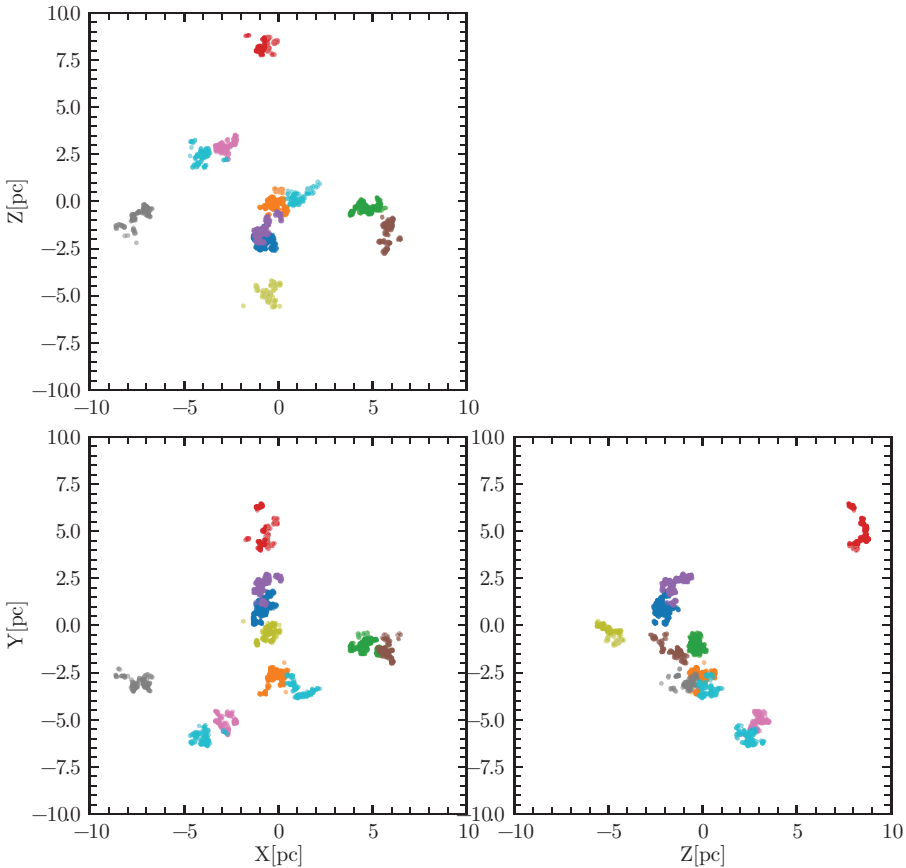


Figure 8.1. Cluster of 1000 stars initially distributed with fractal dimension 1.6 and a half-mass radius 0.5 pc. The 11 different clumps identified by hop are indicated by color. The figure was produced using the script `amuse.examples.plot_fractal_clumpy_cluster`

¹The name derives from the algorithm used to divide the distribution into equivalence classes representing substructure by “hopping” from each particle to its neighbor with the highest local density.

```

58 hop = Hop(unit_converter)
59 hop.particles.add_particles(particles)
60 hop.calculate_densities()
61
62 mean_density = hop.particles.density.mean()
63 hop.parameters.peak_density_threshold = mean_density
64 hop.parameters.saddle_density_threshold = 0.99 * mean_density
65 hop.parameters.outer_density_threshold = 0.01 * mean_density
66
67 hop.do_hop()
68 result = [x.get_intersecting_subset_in(particles) for x in hop.groups()]
69 hop.stop()
70

```

Code example 8.6: Using hop to identify substructure in a particle distribution.

`peak_density_threshold`, the lower limit for recognizing a peak, `saddle_density_threshold`, the phase-space density of a saddle point, and `outer_density_threshold`, which is the minimum density threshold for recognizing a clump. The result in this example is a list of particle sets corresponding to the clumps identified by hop. The algorithm is not particularly fast, but hop is reliable and works intuitively.

8.3.2 The Kepler Package

Plotting positions and animating N -body systems is illustrative and often educational, but not always the most useful for astrophysical analysis. When studying the orbits of planets or multiple systems, the Keplerian orbital elements provide much more quantitative information. These are straightforward to compute from the masses, positions, and velocities of the orbiting bodies, but this is such a common operation that AMUSE contains built-in modules designed to calculate, manipulate, and use Keplerian orbits, and we can simply use one of those to do the work for us. One such module, a legacy of the older `starlab` package,² is simply called `Kepler`.

We begin by initializing the `Kepler` package

```
johannes = Kepler(convert_nbody)
```

where `convert_nbody` is a converter (see Section 2.3). We also generate a particle set containing just the two particles for which we want the orbital elements. For example, in the earlier example on solar-system orbits (Section 3.4.1 and Section 4.4.1), we could call this local particle set `sun_earth` and have its two particles point to the Sun (particle 0) and to Earth (particle 3).

²See github.com/amusecode/Starlab.


```
sun_earth = Particles()
sun_earth.add_particle(particles[0])
sun_earth.add_particle(particles[3])
channel_from_to_sun_earth = gravity.particles.new_channel_to(sun_earth)
```

We also created a local channel to these two particles, which we can use in the event loop to synchronize the gravity code data with the local particles in `sun_earth`.

Within the loop, we reinitialize Kepler with the two particles of interest—Sun and Earth. To print Earth’s semimajor axis, this looks like:

```
johannes.initialize_from_particles(sun_earth)
print(f"Earth's semimajor axis={johannes.get_elements()}")
```

Note that we reinitialize an existing Kepler in the event loop, rather than creating a new one each time. This is done for reasons of efficiency. We could simply create and delete a new Kepler code every time we require these services, but this can incur a considerable overhead in large runs.

Using the Kepler package only for computing simple orbital elements is probably overkill. In AMUSE, the same result can be achieved by calling the routine `orbital_elements_from_binary()`. This function takes two arguments, a particle set (here containing the two stars in a binary) and the gravitational constant `nbody_system.G`, and returns the six orbital elements (see Section 4.4.1). However, the Kepler package can also manipulate orbits—returning particle sets corresponding to a specified time, orbital separation, apocenter, or pericenter—making it a useful tool in many applications. For example, the `multiples` module (Section 8.7.1) uses Kepler to set up scattering experiments, perform two-body orbit integrations, classify scattering outcomes, and rescale particle data.

8.3.3 Recovering from a Code Crash

As discussed in Section 4.6, physics modules may crash if they are placed in regimes for which they were not designed. When a physics module in AMUSE crashes, it is undesirable for the framework also to crash. Using Python exception handling, we can detect when a code has terminated and take appropriate action. We demonstrate this by using a Henyey code to model the evolution of a $10 M_{\odot}$ star until the code crashes. Another example, where we switch from one stellar-evolution code to another, is presented in Section 8.4.3.

Stellar-evolution codes tend to crash at the end of the nuclear fusion as the star approaches its death as a supernova explosion. (It may happen sooner, depending on the code!) Evolving a star to this late stage can be expensive because the stellar-evolution code has a built-in safety valve—if an evolution step fails to converge, the code iteratively reduces the time step and restarts from the previous self-consistent stellar model. This process is repeated until the code has reached a built-in limiting number of restarts, at which point it simply gives up.

```

try:
    while True:
        stellar.evolve_model()
    except AmuseException as ex:
        print(f"Star evolved to t = {stellar.particles[0].age}")

```

Here, we call `evolve_model()` without an argument, which evolves the star using the internal stellar-evolution time step, rather than to a specified time. Once we reach the `print` statement the Henyey code has terminated, but the AMUSE interface has not. We can then obtain the last converged stellar-evolution model, which is as close to an exploding star as the stellar-evolution code can get. In Section 6.4.4, we showed how the postevolution product can be converted into an SPH particle representation to continue the evolution through the supernova explosion.

Rather than using MMAMS, it can be desirable to use a hydrodynamical solver to resolve the collision. A first step in this process would be to convert the one-dimensional stellar-evolution structure from a Henyey code to a realization suitable for continuing the calculation with a hydrodynamics solver. One such routine is `convert_stellar_model_to_sph` from AMUSE's `ext.star_to_sph` package.

In Figure 8.2, we present the density structure for a $1 M_{\odot}$, a $3 M_{\odot}$ and a $10 M_{\odot}$ star, and the associated SPH realization of the same star using 10^4 SPH particles.

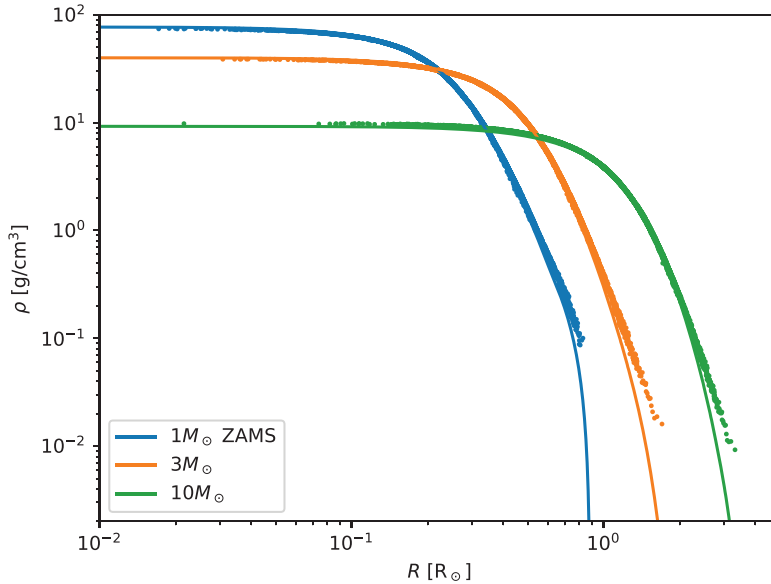


Figure 8.2. Density as a function of distance to the stellar center for three stars of $1 M_{\odot}$, $3 M_{\odot}$ and $10 M_{\odot}$ each at an age of 1 yr. The solid lines show the results from the stellar-evolution model (MESA in this case), the dots give the densities attributed to the SPH particles using the `convert_stellar_model_to_sph` routine. This figure was produced using the script `amuse.examples.single_star_to_sph`, which runs in about an hour. For convenience, the hydro data is also available in `amuse.examples.data.fig_star_in_sph_N10000M1.0MSun.npy` for the $1.0 M_{\odot}$ star, and equivalently for the $3.0 M_{\odot}$ and the $10.0 M_{\odot}$ stars, in which case the plot can be generated using `amuse.examples.plot_single_star_to_sph`.

The SPH realizations nicely follow the density profile calculated using the stellar-evolution code for most of the stellar structure. Near the stellar surface, the density of individual particles exceeds the results from the stellar-evolution code. This is a result of the hydrodynamics solver placing the star in dynamic equilibrium, whereas the stellar-evolution code also requires thermal equilibrium in conjunction with the internal energy source, which is absent in the hydro code. Eventually, near the stellar surface, the hydro code is unable to resolve the density structure in the outermost layers. This latter discrepancy is mainly a resolution issue, with only 10^4 SPH particles, we cannot resolve densities below $\sim \rho_c / \sqrt{10^4}$.

8.4 Elementary Code-coupling Strategies

A straightforward but very important way to couple codes is by directly propagating information from one code to another at the user script level. In this approach, two or more codes work on portions of the data (typically contained in a particle set), which may also be used by other codes. A classic example can be found in star cluster evolution. Stars move due to gravity and at the same time they evolve due to internal nuclear processes. Such a system can be simulated by a straightforward combination of the methods discussed in Chapters 4 and 5, as we now describe.

8.4.1 Combining Gravity with Stellar Evolution

N -body codes have incorporated treatments of stellar evolution for almost four decades. As a consequence, we have by now a fairly good understanding of the numerical subtleties in coupling these pieces of physics. The most common way of solving the combination of stellar evolution and stellar dynamics is the strategy adopted in the seminal work of Terlevich (1987), which updates and synchronizes the two codes at regular time intervals (see also Aarseth 2003)—update the dynamics to some time, then update the stellar evolution to that time, then update the dynamics to the next time using the new stellar masses, etc. We call this *synchronized temporal discretization*.

In general, both codes will have their own internal time steps, which may be different for each object and may change dynamically with time, but in this prescription the time interval for the exchange of information is generally set by the dynamics—typically some fraction of the crossing time of the self-gravitating system. As discussed in Sections 4.1.2 and 5.1.1, this is appropriate in many cases, but it may not be so for systems in resonance, chaotic dynamics, or young massive star clusters, where stellar-evolution time scales may actually be shorter than the relevant dynamical time.

The flexibility afforded by AMUSE allows us to experiment with other time-stepping strategies. One way to improve on the synchronized temporal discretization strategy is to advance the stellar evolution for half a time step, then advance the dynamics for a full step using stellar masses defined at the midpoint, and finally update the stellar evolution to the end of the step. We saw in Section 4.2.3 how evaluating key quantities in the middle of the step leads to improved accuracy. In this case, the stellar evolution cares nothing for the dynamics, but using centered stellar masses improves the accuracy of the dynamical integration. We call this approach *interlaced temporal discretization*, and we explore its use in the next

example. A more complete and elaborate approach to this general problem is presented in Chapter 9.

There are (at least) two complications in coupling stellar evolution with gravitational dynamics: (1) unexpected events in the stellar evolution, and (2) unexpected events in the gravitational dynamics. An example of an unexpected stellar event is a supernova explosion;³ an unexpected dynamical event is a collision between two stars. Neither event can easily be predicted, but they can be handled straightforwardly using interrupts, as discussed in Section A.5.2.

8.4.2 Coupling Radiative Transfer with Hydrodynamics

Combining radiative transport with hydrodynamics is not trivial. In principle, we could adopt a linear coupling strategy between two modules, but a bridge-like approach (see Chapter 9) gives superior results in terms of both better performance and smaller errors. However, since we have not yet discussed a bridge, we adopt a simpler but still effective approach similar to the interlaced temporal discretization discussed in Section 8.4.1.

We have defined a small `RadHydro` class to simplify this function, to be incorporated later into a bridge with (for example) a gravity solver. In order for this class to comply with bridge specifications, it must have a few extra functions: the most important is `evolve_model()`. In addition, to make life easier, we include a method to copy data from the two worker codes to the framework, and one to clean up after we are done. For safety (and convenience), we start by synchronizing all relevant data between the two worker codes and the framework, giving us a self-consistent dataset to work with. For additional safety, albeit probably unnecessary in this example, we perform a similar synchronization after the two worker codes have reached their objectives.

Between the two synchronization operations, we run the hydrodynamical simulation for half a time step, followed by the radiative-transfer simulation for a full time step, and end with a second half time step for the hydrodynamics (see also Section 8.4.1):

```
while t < (tend-timestep/2):
    self.hydro.evolve_model(t+timestep/2.)
    update_rad_from_hydro_channel.copy()
    self.rad.evolve_model(t+timestep)
    update_hydro_from_rad_channel.copy()
    self.hydro.evolve_model(t+timestep)
    t = self.model_time
```

The rest of the radiation–hydrodynamics class is fairly straightforward. In Section 8.6.1, we present an example calculation using this `RadHydro` class using the code example in `amuse.examples.supernova_irradiation`.

³ In practice, since the stellar-evolution codes actually operate one step ahead of the current time, to permit interpolation of their results on demand, a supernova is not completely unexpected. We ignore this operational reality here.

8.4.3 Switching from One Stellar-evolution Code to Another

Stellar-evolution code, in particularly the Henyey codes, can be rather fragile. If such a code becomes impossibly slow, or simply crashes, one would not necessarily stop the entire calculation, but escape to a safer (secondary) stellar-evolution code or prescription. Ideally, the new stellar-evolution code should pick up the failed star from the crashed code and continue its evolution from that same point in its evolutionary stage.

This turns out to be nontrivial because each stellar-evolution code will give a different radius, luminosity, and temperature for the same mass, age, and metallicity. These differences are often small, but can be quite substantial, see for example Section 5.6.1.

To make the transition process simpler, we designed a special module for stellar-evolution codes, allowing us to start with one code and then switch to another at some arbitrary moment. The module is briefly described in Appendix A. The moment to switch codes can be arbitrary but the evolutionary state of where the new stellar-evolution code starts is chosen to be as close as possible to the end-point of the previous code.

The handover algorithm performs an rms search on the relative differences of mass, radius, and luminosity between the last point known state in the primary code and the entire stellar history of the secondary code (here SSE or SeBa) until the star turns into a remnant.

In the following example, we show how a star starts to evolve using Evtwin, for 1 Myr, and then we switch (for no particular reason) over to SSE. The interface function `EVtwin2SSE()`, is really a small class with two stellar-evolution codes and a collection of other parameters. One of these parameters is `EVtwinAgeAtSwitch`, indicating at what time to switch from one code to the other. Another parameter is `EVtwinException`, which is used if the code-switch is nonvoluntary (i.e., the primary stellar-evolution code crashed).

```
from amuse.lab import Particle
from amuse.lab import units
from amuse.community.evtwin2sse.interface import EVtwin2SSE

stellar = EVtwin2SSE()
stellar.EVtwinAgeAtSwitch = 1 |units.Myr
star.mass = 10|units.MSun
stellar.particles.add_particle(star)

print(stellar.particles)
stellar.evolve_model(10|units.Myr)
print(stellar.particles)
```

8.4.4 Event-driven Simulations

So far, our simulations of star clusters have alternately called the gravitational dynamical evolution and stellar-evolution solvers at regular time intervals. This gives a somewhat limited perspective on reality—a massive star does not wait until a

dynamical encounter is over before blowing up, and a stellar collision does not wait until the end of a stellar-evolution step. Instead, each may interrupt the otherwise orderly joint evolution process.

In conventional simulation languages, taking these interrupts into account at the proper times is difficult—the language semantics simply are not flexible enough to deal with these events. In AMUSE, an event-driven simulation can be easily realized with the proper use of timing, channels, declarations, and calls. In Code example 8.7, we create data structures and declare two codes, a stellar dynamics solver and a stellar-evolution code, together with communication channels. In principle, there is nothing really new about these, except that both codes now have stopping conditions enabled.

Code example 8.8 shows the event loop, which starts by querying the stellar-evolution code about the shortest safe time step, the minimum interval during which no star will experience a supernova or significant changes to its `stellar_type` (see Appendix A). The collision handling for the event-driven simulation is handled by a separate function (see the example script, Code example 8.4). The supernova handling is illustrated in Code example 8.9. Strictly speaking, this simulation is only event-driven in terms of the collision handling. The stellar evolution is

```

106 while time < t_end:
107
108     dt_min = min(dt, stellar.particles.time_step.min())
109     print("Time steps:", dt.in(units.Myr), dt_min.in(units.Myr))
110     time += dt_min
111
112     bodies.radius *= 1.e+5
113     channel_to_gd.copy_attributes(["mass", "radius", "vx", "vy", "vz"])
114     E_dyn = gravity.kinetic_energy + gravity.potential_energy
115     gravity.evolve_model(time)
116     dE_dyn = E_dyn - (gravity.kinetic_energy + gravity.potential_energy)
117
118     resolve_collision(collision_detection, gravity, stellar, bodies)
119     channel_from_gd.copy()
120
121     time = gravity.model_time
122
123     E_stellar = gravity.kinetic_energy + gravity.potential_energy
124     stellar.evolve_model(time)
125     dE_stellar = E_stellar - (gravity.kinetic_energy
126                             + gravity.potential_energy)
127
128     resolve_supernova(supernova_detection, bodies, time)
129     channel_from_se.copy_attributes(["mass", "radius", "age",
130                                     "temperature", "luminosity"])
131
132     write_set_to_file(bodies.savepoint(time), filename, 'hdf5')
133     print_diagnostics(time, bodies.mass.sum(), E_dyn,
134                       dE_dyn, dE_coll, dE_stellar)

```

Code example 8.8: Event-driven simulation with collisions and supernovae.

```

41 def resolve_supernova(supernova_detection, bodies, time):
42     if supernova_detection.is_set():
43         print("At time=", time.in_(units.Myr), \
44               len(supernova_detection.particles(0)), 'supernova(e) detected')
45
46     Nsn = 0
47     for ci in range(len(supernova_detection.particles(0))):
48         print(supernova_detection.particles(0))
49         particles_in_supernova \
50             = Particles(particles=supernova_detection.particles(0))
51         natal_kick_x = particles_in_supernova.natal_kick_x
52         natal_kick_y = particles_in_supernova.natal_kick_y
53         natal_kick_z = particles_in_supernova.natal_kick_z
54
55         particles_in_supernova \
56             = particles_in_supernova.get_intersecting_subset_in(bodies)
57         particles_in_supernova.vx += natal_kick_x
58         particles_in_supernova.vy += natal_kick_y
59         particles_in_supernova.vz += natal_kick_z
60         Nsn += 1
61
62     print('Resolved', Nsn, 'supernova(e)')

```

Code example 8.9: Function for resolving a supernova event.

predictable so long as there are no additional complications; for example, due to mass transfer in a close binary system.

We run the gravitational dynamics for that safe time frame. If a collision occurs during this time, the N -body code is interrupted and the encounter is resolved, ending the dynamics step. In Code example 8.8, we ensure that the stars are evolved to the same time as the N -body integrator. For technical reasons, individual time step codes do not normally guarantee to evolve the system to exactly the time given in the call to the community code. This can be overruled by forcing the N -body code to synchronize to the target time with

```
gravity.parameters.force_sync = True
```

We normally would not recommend synchronizing an N -body code in this way, but it is often necessary in multiscale and multiphysics problems. After the dynamics step, the stellar evolution is updated to the current age of the system, which is the moment of a collision, the moment of a supernova, or some predetermined output time. In Section 10.1.3, we give an example of `force_sync` in the `venice` module in AMUSE.

The results of such an event-driven simulation can be distinctly different from a regular calculation in which both modules are called at fixed time intervals. This is demonstrated in Figure 8.3, where we compare three simulations, one without stellar evolution, and one with constant time steps between stellar dynamics and stellar evolution. The third simulation (green curve in Figure 8.3) is event-driven. In this simulation there was no real chance of stellar collisions because the initial cluster was quite large, so we followed a slightly different event-driven strategy: we evolved

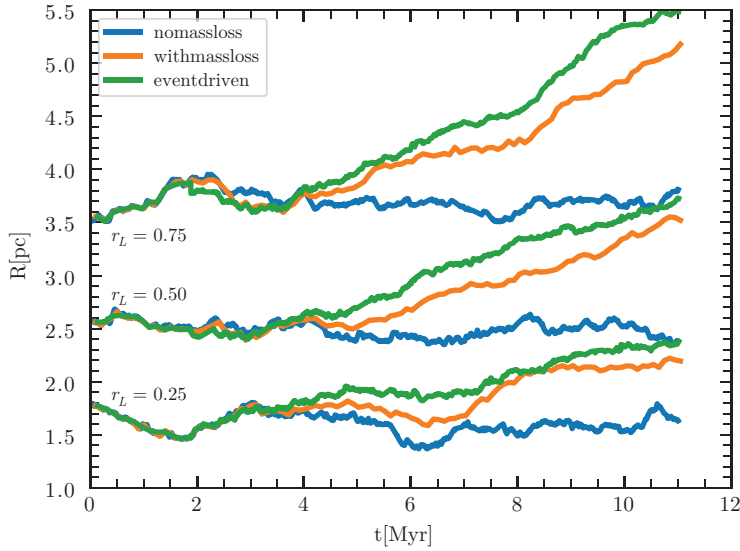


Figure 8.3. Lagrangian radii containing 25%, 50%, and 75% of the total cluster mass for three simulations of a cluster of 1000 stars. Each simulation is performed using exactly the same initial realization. In one of these runs (blue curve), stellar mass loss was ignored. In the second (orange curve), mass loss was applied at constant time intervals of 0.1 Myr after the dynamics step (note that in Figure 8.4 we adopted a very small time step of only 10% of the inner orbital period). The green curve is the result of an event-driven simulation (see Section 8.4.4), and should, therefore, be considered most reliable. The figure was produced using the script `amuse.examples.gravity_stellar_comparison`, which should run in 5–10 minutes on a laptop.

all stars for a minimum stellar-evolution step, followed by the dynamics for the same amount of time.

Figure 8.3 shows the evolution of the 25%, 50%, and 75% Lagrangian radii (see Section 4.1.3.1) of three initially identical clusters of 1000 stars over a time interval of 10 Myr. The stars were initially distributed as a $W_0 = 3$ King model (King 1966, see Section 3.2), with stellar masses drawn from a Salpeter mass function between $1 M_\odot$ and $100 M_\odot$ (see Section 3.3). All runs were performed using the same dynamical integrator. The differences in the Lagrangian radii are therefore a consequence of how stellar mass loss propagates into the dynamical evolution of the cluster.

In the cluster without stellar mass loss, the Lagrangian radii shrink slightly over the 10 Myr period studied, since pure dynamics dictates the evolution. During the first ~ 3 Myr, the orange and blue (and green) clusters show basically the same dynamical evolution. The differences only become evident later, when the most massive stars start to lose significant mass. After ~ 4 Myr, the Lagrangian radii of the runs with stellar mass loss start to expand.

In the cases with mass loss, about 18% of the initial cluster mass of $3200 M_\odot$ is lost, causing the 25%, 50%, and 75% Lagrangian radii to expand by 30–50%. The outermost Lagrangian radii expand slightly faster than the more inner Lagrangian radii. These expansion factors are significantly larger than would be naively expected based on an adiabatic response of the cluster, which would predict an 18% increase

in size (Carathéodory 1909). The reason is that much of the mass lost is ejected impulsively by the strong winds of the massive Wolf–Rayet stars and (after about 3 Myr) supernova. In addition, the orbits of the mass-losing stars are not circular, as is assumed in the naive adiabatic argument, but are generally quite eccentric.

At the start of these particular simulations, none of the 10 most massive stars ($\gtrsim 20 M_{\odot}$) lay within the inner 25th percentile. By the end of the simulation, at 10 Myr, this number increased to four in the run without stellar evolution, and two in the other cases. Three massive stars were initialized outside the 75th percentile and had insufficient time to complete an orbit around the cluster center. The more rapid expansion of the outermost Lagrangian radii compared with the innermost radii is therefore in part due to the mass lost by these more massive stars, and not by the segregation of the most massive stars to the cluster center.

Event-driven simulations are generally somewhat slower than fixed-time interaction simulations, but they are probably worth the cost because the physics is better represented by the numerical model.

8.4.5 Evolution of a Hierarchical Triple System

In this section, we demonstrate an interlaced coupling between stellar evolution and gravitational dynamics by integrating a triple composed of three massive stars, each of them losing mass copiously. The orbital evolution of such a system cannot be described analytically, so the problem must be addressed numerically. This problem turns out to be rather hard because the various time scales of the problem are comparable. This example is not much different than the one presented in Section 8.4.4, but here we apply it to a tightly coupled triple system. Of course, one could consider performing this simulation with the more dedicated triple evolution code *Tres*, discussed in Section B.2.1.

The triple system θ Muscae comprises the WC5/6 Wolf–Rayet star WR48 on an inner orbit with period $P_{\text{in}} \simeq 19$ days around an O6/7V star, with an O9.5/B0Iab companion on a much wider outer orbit (Hill et al. 2002; Sugawara et al. 2008). The other orbital parameters are not known, but each of the three stars has a strong stellar wind. We can easily integrate the equations of motion of such a triple system within AMUSE, taking the mass loss of each star into account, by coupling a stellar-evolution code with an N -body integrator.

We adopt masses at birth of $60 M_{\odot}$ (for WR48) and $30 M_{\odot}$ for the inner two binary stars, and $20 M_{\odot}$ for the outer companion. The nature of the Wolf–Rayet star suggests that the system is at least a few megayears old. We therefore began the simulation by evolving the three stars to an age of 4.5 Myr using *SeBa*. By that time, the masses of the three stars had fallen to $12.48 M_{\odot}$, $28.52 M_{\odot}$, and $19.81 M_{\odot}$. It is difficult to judge whether or not these masses are consistent with the observations, but at least their spectral types are. The mass-loss rates at that age are $-1.6 \times 10^{-5} M_{\odot} \text{ yr}^{-1}$, $-4.2 \times 10^{-7} M_{\odot} \text{ yr}^{-1}$, and $-5.2 \times 10^{-8} M_{\odot} \text{ yr}^{-1}$. Further details on the stellar parameters are presented in Table 8.1.

After this initial period of stellar evolution, we initialized the N -body code (in this case *Huayno*) in a configuration consistent with the observed orbit. We assumed

Table 8.1. Adopted Parameters for the Stellar Wind Binary θ Muscae

Parameter	Unit	Star 1	Star 2	Star 3
Spectral type		WR5	O6V	O9.5III
Mass	$M_{\odot}$	12	20	30
Radius	$R_{\odot}$	2.2	10	50
Luminosity	$L_{\odot}$	2.2×10^5	1.4×10^5	0.55×10^5
Temperature	K	7.1×10^4	4.5×10^4	3.9×10^4
Mass-loss rate	$M_{\odot} \text{ yr}^{-1}$	1.8×10^{-5}	1.0×10^{-6}	1.0×10^{-6}
Wind velocity	km s^{-1}	2000	1000	1000

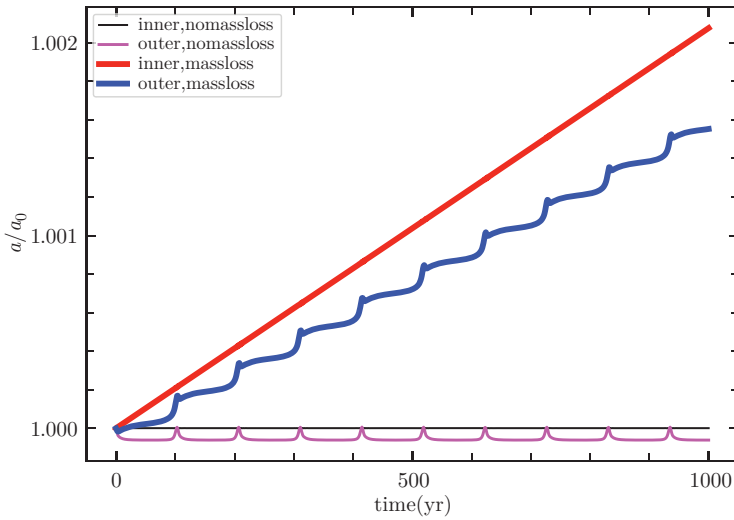


Figure 8.4. Evolution of the orbital semimajor axes of the inner (blue) and outer (red) orbits of the hierarchical triple system θ Muscae. All quantities are expressed relative to their initial values (at $t = 4.5$ Gyr). At the start of the integration, at 4.5 Gyr, the masses were 12.48, 28.52, and 19.81 $M_{\odot}$, respectively. The integration spanned a time interval of 10^3 yr. The script to generate the figure, `amuse.examples.plot_triple_with_wind`, takes a few minutes to run. The code that generated the data is available in `amuse.examples.evolve_triple_with_wind`.

that the outer orbit has an inclination of 30° relative to the inner one, with an inner eccentricity of 0.2, and an outer eccentricity of 0.6, although these are not known from observations. For definiteness, we chose an initial mean anomaly and argument of periastron of 180° , and an outer orbital separation of 100 AU.

Figure 8.4 presents the orbital semimajor axes and eccentricities of the inner and outer binaries during an integration from 4.0 Myr to 4.55 Myr using the interlaced scheme just described. The Wolf–Rayet star exploded as a supernova shortly after this, at an age of 4.6 Myr. During the integration, we continued to evolve the stars using SeBa, and the mass loss was propagated to the N -body integrator after every

$0.1 P_{\text{in}} = 1.9$ days. The evolution of θ Muscae over the relatively small time frame presented in Figure 8.4 barely requires the use of a stellar-evolution code, but it allows us to illustrate how stellar evolution can be combined with the dynamics of this complicated triple system.

The orbits of both binaries expanded due to mass loss. On the timescale of the integration, the inner two stars lost $0.18 M_{\odot}$, whereas the outer star lost $0.026 M_{\odot}$. From Figure 8.4 we can see that the outer binary expanded by about 0.8%, which is about a factor of two more than the fractional mass loss from all stars combined. The inner binary was much more strongly affected by mass loss, and expanded by about 2.7%. The variation in the eccentricity of the inner binary is caused by its interaction with the outer star. The classical Lidov–Kozai (von Zeipel 1910; Lidov 1962; Kozai 1962) period is about 0.44 Myr, and corresponds to the large excursions in the eccentricity of the inner binary. The eccentricity of the outer binary remained almost constant.

In this example, and in fact in all other current approaches, mass is lost in small chunks at fixed time intervals. This causes the system to receive a small shock after every mass-loss step, and this may have undesirable side effects. In fact, in constructing Figure 8.4, we found that the outcome of the simulation is remarkably sensitive to the choice of integration parameters, especially the time interval on which stellar mass is applied. The script used to perform this calculation can be found in `amuse.examples.evolve_triple_with_wind`, and we invite readers to explore its outcome with different choices for various parameters. It will take between a few minutes and many days to run, depending on the parameter values chosen; the parameters leading to Figure 8.4 took a few hours on a laptop. This sensitivity is a topic of ongoing research, and AMUSE modules capable of incorporating mass loss directly into the dynamical integration are currently under development.⁴ While these are clearly counter to the AMUSE philosophy of not making major changes to the internals of community modules, sometimes code validation requires more intrusive measures.

The relatively poor performance of the code in this particular strongly coupled problem motivated us to develop a new symplectic N -body code with the possibility of including a mass-loss term in the integrator. We call this code `symple` for its simple structure, its simple implementation, and the symplecticity of the adopted integrator. In fact, `symple` can be run with a wide range of integrators of different order ranging from first order to 10th order, each of which is symplectic. For more details about `symple`, we refer to Section A.3.4.

Although it goes well beyond the scope of this chapter, as a preview of AMUSE capabilities introduced in Chapter 6 and soon in Chapter 9, Figure 8.5 shows an image of the wind produced in θ Muscae, where now we integrate the equations of motion of the individual stars and simultaneously resolve the winds using a hydrodynamical simulation (Jänes et al. 2014). Specifically, the simulation shows

⁴See `AMUSE_updates.pdf` for ongoing developments.

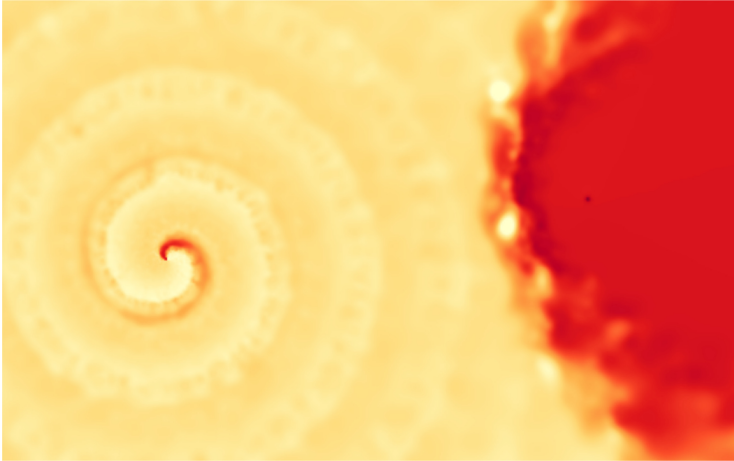


Figure 8.5. Evolution of the Wolf-Rayet triple-star system θ Muscae using `Huayno` (Jänes et al. 2014) as the gravity solver and `fi` for the hydrodynamics (see Chapter 6). The hydrodynamics was resolved using the stellar wind routines in `AMUSE` (see van der Helm et al. 2018). The script used to produce this figure is too long (and takes too long to run) to reproduce in the text, but its basic principles are described in Chapters 6 and 9.

the gas density in the orbital plane of the hierarchical triple system. While the winds collide, the three stars orbit each other, resulting in the tight spiral (left) from the inner binary that interacts with the massive wind of the tertiary outermost star. The simulation was performed using the SPH code `Fi` using a mass of $10^{-11} M_{\odot}$ for each SPH particle. Both the inner and the outer binaries are initially circular and in the same plane with orbital period of $P_{\text{in}} = 19.1$ days and $P_{\text{out}} = 130$ years.

The wind was resolved in the hydrodynamics solver using the `AcceleratingWind` routines from `amuse.ext.stellar_wind`, which produces a wind acceleration inversely proportional to the distance squared (or $\propto 1/r^2$). The results of the calculations are presented in Figure 8.5.

Evidently, our “simple” example of integrating a triple-star system has already proved quite challenging. What about an entire star cluster? Generating initial conditions for a star cluster in which we integrate the equations of motion and simultaneously evolve the stars is basically the same issue as discussed in Chapter 4 and 5. Once the initial realization has been created, we can start the stellar-evolution code and the gravity solver in much the same way as we did here for θ Muscae.

8.5 Stellar Cluster with Collisions

8.5.1 Sticky Stars

To demonstrate the working of stopping conditions and collision handling in direct N -body codes, we present an example of a simple cluster in which stars are allowed to collide. We opted for an $N = 1000$ body system initially distributed as a $W_0 = 7$ King model with a Salpeter mass distribution between $0.1 M_{\odot}$ and $10 M_{\odot}$ (see Section 3.3). Each star was given a radius of 0.001 N -body length units, which

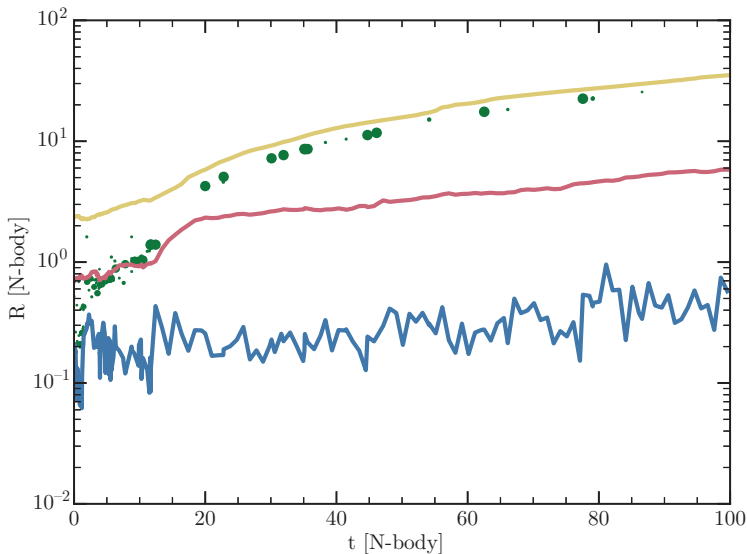


Figure 8.6. Direct N -body simulation (using `ph4`) of a small star cluster over 100 N -body time units. Initially, the cluster contained $N = 1000$ stars with a Salpeter mass distribution spanning two orders of magnitude in mass, distributed in a virialized King model with $W_0 = 7$. The initial radii of all stars were taken to be 0.001 N -body length unit. The bullets indicate the times and locations of stellar collisions; the size of a bullet indicates the mass of the collision product. We assume that no mass is lost and the mean stellar density remains constant. The three curves (bottom to top) illustrate the core radius (blue), the half-mass radius (red), and the 90% Lagrangian radius (yellow) of the cluster. The script that produced this figure is `amuse.examples.gravity_collision`. It should take a few minutes to run on a laptop.

resulted in many interesting collisions. The code example is too long to reproduce here, but it is available in `amuse.examples.gravity_collision`.

We ran one simulation with virialized initial conditions. Collisions were resolved using the stopping conditions described in Section A.5.2. For collision products, we assumed no mass loss and calculated a new radius such that the mean density remained constant. The integration was performed using the direct integrator `ph4`. Stellar evolution was not included (see Section 5.1). Figure 8.6 presents the evolution of the core and the half-mass radii of the N -body system; the moments at which collisions occur are indicated by bullets. The size of a bullet is proportional to the mass of the collision product.

8.5.2 Using a Separate Code to Manage a Collision

If instead of running a parameterized stellar-evolution code we are using a full Henyey code, such as MESA or EVtwin, we could in principle use the detailed structural information from the stellar-evolution code to construct three-dimensional models of the colliding stars and model the collision as a hydrodynamical simulation, as we will discuss later in Section 8.5.4. This should, in principle, give the most accurate estimate of mass loss, energy released, etc., but it is typically a computationally expensive calculation—perhaps too expensive to be done routinely.

A useful intermediate solution to the merger problem is provided by the MMAMS module (Lombardi et al. 2003; Gaburov et al. 2008), as described in Section 5.2.9. The script presented earlier in Code example 5.5 can be used to determine the structure of the merger product. The only additional actions needed are to reposition the merger product in the center-of-mass frame of the two colliding stars, and to conserve momentum, as in Code example 8.5.

8.5.3 Restarting Stellar Evolution after a Merger

After the collision has been detected and managed, the evolution of the merger product can be continued. For a parameterized code, we simply use a rejuvenation formula to place the merged star on a new track at some evolutionary time (see also Section 5.3.12). We do have a script to transfer the stellar evolution from a parametrized code to a full Henyey code, which is an example in the AMUSE repository called `hr_diagram_cluster`.

However, if we are using a Henyey code, we can continue the detailed evolution of the merger product. Code example 8.10 shows how this can be done. In this example, we evolve two stars to the same time using MESA, merge them using MMAMS, and then continue the evolution of the merger product using MESA.

Figure 8.7 presents (yellow dashed line) the evolutionary track of the collision product resulting from the merger of stars of masses $3 M_{\odot}$ and $1 M_{\odot}$. The stars collided at 150 Myr, which is half the main-sequence lifetime of the more massive star (determined by first evolving the primary to the end of its nuclear burning lifetime using EVTwin). For comparison, we overplot the evolutionary tracks of single stars of masses $3 M_{\odot}$ and $4 M_{\odot}$, as well as the track for a main-sequence star of mass $3.70 M_{\odot}$ (the value returned by MMAMS).

The evolutionary track of the merger product makes several rapid excursions in temperature and luminosity before settling after $\sim 10^5$ yr (comparable to its Kelvin–Helmholtz timescale, see Equation (5.6)) onto a new “rejuvenated” track, after which its evolution continues along the main sequence at a slightly lower luminosity than the $4 M_{\odot}$ star. The merger evolutionary track differs significantly from that of a canonical $3.70 M_{\odot}$ star, mainly due to its nonstandard abundance profile. Subsequently, on the subgiant and giant branches, the track is quite similar to that of the $4 M_{\odot}$ star.

In many cases, a simplified parameterized prescription for the consequences of a stellar merger may be quite satisfactory, although not very accurate. One can ask whether spending significantly more computer time on this complex problem is worth the effort, but many studies have demonstrated that detailed modeling of stellar collisions can give results quite different from the simplified semianalytic prescriptions described in Section 5.3.12 (Lombardi et al. 1995, 2002, 2003).

8.5.4 Using a Hydro Code to Simulate a Stellar Merger

In Section 8.5.2, we demonstrated how to use a separate code to resolve a stellar collision using a parameterization of the collision process. Instead of a

```

71 def merge_two_stars_and_evolve(
72     mass_primary, mass_secondary, time_collision, time_end
73 ):
74     stars = Particles(2)
75     stars.mass = [
76         mass_primary.value_in(units.MSun),
77         mass_secondary.value_in(units.MSun)
78     ] | units.MSun
79     stellar_evolution = Mesa(**MESA_OPTIONS)
80     stellar_evolution.parameters.metallicity = 0.0142
81     stellar_evolution.particles.add_particles(stars)
82     while stellar_evolution.model_time < time_collision:
83         stellar_evolution.evolve_model()
84         print_stars(stellar_evolution)
85     n_shell = min(
86         stellar_evolution.particles[0].get_number_of_zones(),
87         stellar_evolution.particles[1].get_number_of_zones(),
88     )
89     merger_code = Mmams(**MMAMS_OPTIONS)
90     merger_code.parameters.target_n_shells = n_shell
91     merger_code.parameters.dump_mixed_flag = True
92     merger_code.parameters.do_shock_heating_flag = True
93     merger_code.commit_parameters()
94     handler = CollisionHandler(
95         merger_code,
96         stellar_evolution_code=stellar_evolution,
97     )
98     merger_product = handler.handle_collision(
99         stellar_evolution.particles[0],
100         stellar_evolution.particles[1],
101     )
102     merged = stellar_evolution.particles[0]
103     stellar_evolution.evolve_model(keep_synchronous=True)
104     p = stellar_evolution.particles[0]
105     while stellar_evolution.model_time < time_end:
106         stellar_evolution.evolve_model()
107         print(
108             f"Time={stellar_evolution.model_time} {p.stellar_type} "
109             f"{p.mass} {p.radius} {p.temperature} {p.luminosity}"
110         )
111         if p.stellar_type >= 4 | units.stellar_type:
112             break
113     merger_code.stop()
114     stellar_evolution.stop()

```

Code example 8.10: Merge two stars and evolve the merger product. The code example is available in `amuse.examples.merge_two_stars_and_evolve`. Several lines related to plotting Figure 8.7 have been omitted.

parameterized code, we could also use a hydrodynamics solver for the collision. Here, we do just that, using an SPH code to resolve the merger.

8.5.4.1 Generating Stellar Structure in a Particle Set

To start the merger simulation, we first need the structure of two stars. We saw in Chapter 5 how this can be realized. Code example 8.11 demonstrates the process again.

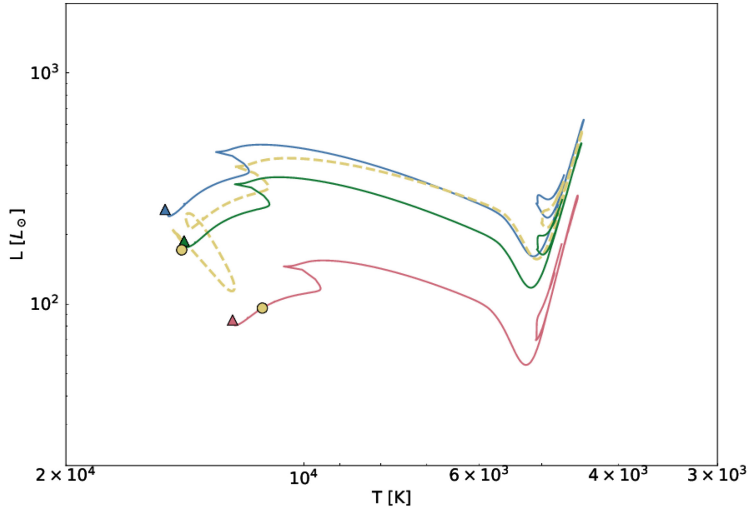


Figure 8.7. Evolutionary tracks for a $3 M_{\odot}$ star (red curve) a $4 M_{\odot}$ star (blue), and a $3.70 M_{\odot}$ star (green) from the main sequence (filled triangles) to the bottom of the AGB. The yellow dashed curve shows the evolutionary track of the merger product of a $3 M_{\odot}$ star and a $1 M_{\odot}$ star that collide halfway along the primary’s main-sequence track, with the merger structure determined by MMAMS. The yellow circles represent the evolutionary states of the primary immediately before the collision and of the merger product immediately afterward. This figure was produced using the script `amuse.examples.merge_two_stars_and_evolve`, which should run in 10–15 minutes on a laptop.

```

17 def return_evolved_star_hydro(mass, time, number_of_sph_particles):
18     star = Particle(mass=mass)
19     stellar = Evtwin()
20     star = stellar.particles.add_particle(star)
21     stellar.evolve_model(time)
22     number_of_sph_particles = number_of_sph_particles * int(mass.value_in(units.MSun))
23     star_in_sph = convert_stellar_model_to_sph(
24         star, number_of_sph_particles
25     ).gas_particles
26     stellar.stop()
27     return star_in_sph

```

Code example 8.11: Using a Henyey code to construct an SPH realization of a single star of mass `mass` evolved to time `time`. The complete code example is in `amuse.examples.merge_two_stars_sph`.

In addition to evolving the star to the desired time, this code also converts the stellar structure provided by the Henyey code to an SPH particle representation:

```
star_in_sph = convert_stellar_model_to_SPH(star, N_sph).gas_particles
```

The method `convert_stellar_model_to_SPH` returns a Python class having `gas_particles` as an attribute. We run the same procedure for the primary and the secondary stars, then add the two SPH realizations together after offsetting the secondary with respect to the primary and giving them some relative velocity. This is


```

34 def merge_two_stars_sph(
35     mass_primary, mass_secondary, time_collision, number_of_sph_particles
36 ):
37     primary_in_sph = return_evolved_star_hydro(
38         mass_primary, time_collision, number_of_sph_particles
39     )
40     primary_in_sph = relax_sph_realization(primary_in_sph)
41     secondary_in_sph = return_evolved_star_hydro(
42         mass_secondary, time_collision, number_of_sph_particles
43     )
44     secondary_in_sph = relax_sph_realization(secondary_in_sph)
45     radius = primary_in_sph.x.max() + secondary_in_sph.x.max()
46     total_mass = primary_in_sph.mass.sum() + secondary_in_sph.mass.sum()
47     secondary_in_sph.x += 0.8 * radius
48     secondary_in_sph.y += 0.6 * radius
49     secondary_in_sph.vx -= (constants.G * total_mass / radius).sqrt()
50
51     converter = nbody_system.nbody_to_si(mass_primary, 1.0 | units.au)
52     hydro = Gadget2(converter)
53     hydro.gas_particles.add_particles(primary_in_sph)
54     hydro.gas_particles.add_particles(secondary_in_sph)
55     hydro.evolve_model(2.0 | units.hour)
56     hydro.gas_particles.new_channel_to(primary_in_sph).copy()
57     hydro.gas_particles.new_channel_to(secondary_in_sph).copy()
58     hydro.stop()
59     return primary_in_sph, secondary_in_sph

```

Code example 8.12: Using an SPH code to merge two stars. For the full code example see `amuse.examples.merge_two_stars_sph`.

illustrated in Code example 8.12. The script looks similar to Code example 5.5, except that here we convert the evolved stars into N smoothed gas particles.

The conversion from Henyey to SPH is done on a grid, which introduces artifacts and generally leaves the SPH star slightly out of equilibrium. We address this problem by relaxing the SPH models (in this case, using a separate SPH code), as shown in Code example 8.13. Here, we slowly reduce the velocities of the SPH particles while running the hydrodynamics code with short steps. This is the same process as for creating a “glass” solution for initial conditions as is done in many simulations (see, e.g., Section 3.3 on page 40 in White 1994). In this example, we have opted to relax the star for 2.5 dynamical times in 250 steps per dynamical timescale, which is rather short. It is better to have the star relax for far longer: J. Lombardi (2010, private communication) recommends at least 25 dynamical time scales.

By default, the output of `convert_stellar_model_to_SPH` represents a star at rest at the origin. Hence, before combining the two SPH realizations of the stars in a hydro solver, their positions and velocities must be changed to reflect the initial condition of the collision. This can be done by shifting each of the stars to its desired center-of-mass position, with all particles within each star being given the same velocity. In Code example 8.12, we position the secondary star such that it is almost touching the primary star, with a transverse velocity of roughly the escape speed in a direction that results in a grazing collision.

```

66 def relax_sph_realization(sph_star):
67     dynamical_timescale = sph_star.dynamical_timescale()
68     converter = nbody_system.nbody_to_si(dynamical_timescale, 1 | units.RSun)
69     hydro = Gadget2(converter, number_of_workers=2)
70     hydro.gas_particles.add_particles(sph_star)
71
72     to_framework = hydro.gas_particles.new_channel_to(sph_star)
73
74     ts_factor = 2.5
75     time_end = ts_factor * sph_star.dynamical_timescale(mass_fraction=0.9)
76     n_steps = ts_factor * 100
77     velocity_damp_factor = 1.0 - (ts_factor * 2 * np.pi) / n_steps
78     time_step = time_end / float(n_steps)
79     time = 0 | units.day
80     while time < time_end:
81         time += time_step
82         hydro.evolve_model(time)
83         hydro.gas_particles.velocity = (
84             velocity_damp_factor * hydro.gas_particles.velocity
85         )
86     to_framework.copy()
87     hydro.stop()
88     return sph_star

```

Code example 8.13: Relaxing the SPH realization of a star in order to prepare it for a hydrodynamical collision simulation. This code fragment is from `amuse.examples.merge_two_stars_sph`. Here, we introduced a scaling factor `ts_factor = 2.5`. This factor indicates how slowly (or how quickly) we reduce the velocities of the particles.

In order to prevent the two stars from moving to the left due to the momentum added to the system, one could move all the SPH particles to their center-of-mass frame after merging,

```

all_sph_particles = ParticlesSuperset([primary_in_sph, secondary_in_sph])
all_sph_particles.move_to_center()

```

Given that SPH is Galilean-invariant, the shift to the center-of-mass frame is unnecessary, but it makes graphical presentations of the encounter much more convenient.

8.5.4.2 Resolving the Collision with the Hydrodynamics Solver

With the initial conditions established, we instantiate the hydrodynamics solver `Gadget2`

```
hydro = Gadget2()
```

and add the SPH particle sets of the two stars to the community code:

```

hydro.gas_particles.add_particles(primary_in_sph)
hydro.gas_particles.add_particles(secondary_in_sph)

```

Here, we add the particle sets for each of the two stars separately, in order to keep track of which star each SPH particle initially belonged to.

In Code example 8.12, we run the hydrodynamics code for a simulation time of only two hours without saving any diagnostic data. For production simulations, this is not desirable. In many cases, it is better to run the specific code in a loop over time, testing conservation of energy and angular momentum, printing some diagnostics, and writing the raw data to a file at the end of each step. Such a loop might look like

```
time_end = 10 | units.hour
time_step = 1 | units.hour
while hydro.model_time < time_end:
    hydro.evolve_model(hydro.model_time + time_step)
    channel_from_hydro_to_framework.copy()
    print_diagnostics(hydro)
    write_set_to_file(primary_in_sph, "hydro.amuse")
    write_set_to_file(secondary_in_sph, "hydro.amuse")
hydro.stop()
```

Here, `print_diagnostics(hydro)` might print time, energy, angular momentum, and other useful diagnostic information reflecting the progress of the simulation.

8.5.4.3 Recognizing a Star in the Collision Product

It is not completely trivial to recognize the stellar merger product in the messy hydrodynamical blob that results from a collision. Such collisions may be highly asymmetric, rotate rapidly, and have a giant envelope of barely bound low-density gas. Figure 8.8 demonstrates the asymmetries one can expect from a rather violent collision, in this case between a $80 M_{\odot}$ and a $20 M_{\odot}$ star, both at an age of about 0.49 Myr. In this calculation, we adopted Gadget2 to perform the hydrodynamical simulation using 10^5 equal-mass SPH particles. The amount of mixing seen in Figure 8.8 is somewhat surprising when compared to earlier results, particularly because such collisions tend to lead to relatively little mixing (see the discussion in Lombardi et al. 1996). The use of more realistic stellar structure models cannot be the reason for the discrepancy (see Sills & Lombardi 1997), but it is possible that the very high stellar masses adopted, compared to the initial conditions in Lombardi et al. (1996), could lead to more mixing.

Here, we adopt a rather simple approach to the problem of identifying the SPH particles making up the merger product. We analyze the results of the SPH simulation using the hop clump finder (see Section 8.3.1). The three control parameters for peak density threshold, saddle point density cutoff, and the outer density threshold are specified to help hop decide which particles belong to a coherent structure. Each of the groups in the list of particle sets (`hop.groups`) identified by hop belongs to an identified structure. For our stellar merger calculation, we are interested in the largest one.

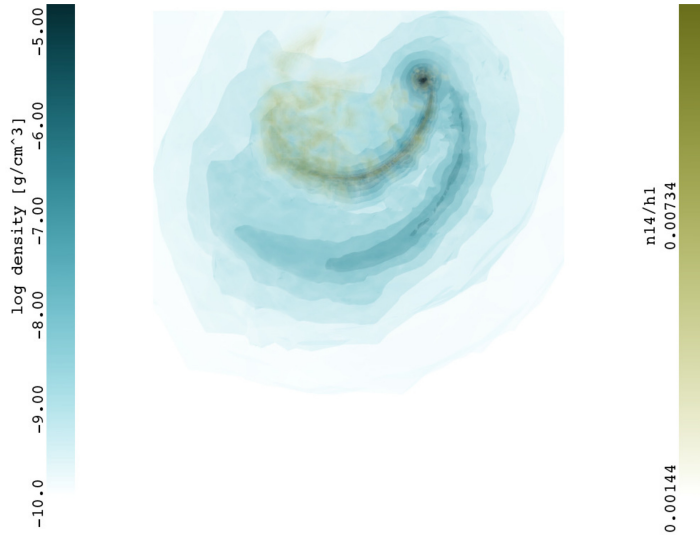


Figure 8.8. Projected density distribution of the gas ejected shortly after the merger between two massive stars. The results of the hydrodynamical simulation are shown 8.8 days after the merger between a $M_{\text{in}} = 80 M_{\odot}$ star and a $m_{\text{in}} = 20 M_{\odot}$ star in an $a_{\text{in}} = 77 R_{\odot}$ orbit with an eccentricity of $e_{\text{in}}^B = 0.87$. Projected density is shown in blue, while the nitrogen/hydrogen ratio is presented in mustard. The origin is at the center of mass of the original two-body system. The two massive loops of ejected gas (at left) are characteristic of a merger. We do not present the plotting routine for this figure, but instead refer to Portegies Zwart & van den Heuvel (2016) for more details. Adapted with permission from (Portegies Zwart & van den Heuvel, 2016). © 2016 The Authors. Published by Oxford University Press on behalf of the Royal Astronomical Society.

```
def find_clumps_with_hop(sph_particles, unit_converter):
    hop = Hop(unit_converter)
    hop.particles.add_particles(sph_particles)
    hop.calculate_densities()

    mean_density = hop.particles.density.mean()
    hop.parameters.peak_density_threshold = mean_density
    hop.parameters.saddle_density_threshold = 0.99*mean_density
    hop.parameters.outer_density_threshold = 0.05*mean_density
    hop.do_hop()

    blob = [x.get_intersecting_subset_in(sph_particles) for x in hop.groups()]
    hop.stop()
    return blob
```

Finding clumps in an SPH simulation is commonly done in postprocessing, but it can also be used to monitor the progress of the merger simulation. In the simulation presented in Figure 8.8, we monitored the clumpiness of the particle distribution at run time, and continued until only a single blob of particles remained. After the detection of this final blob, we ran the hydrodynamics for a further 5.5 years to ensure that it had come into dynamical equilibrium. This final blob is considered to be the merger product, and the residual material is identified as ejecta from the merger process. In this simulation, the merger resulted in a single $83 M_{\odot}$ star.

Once coherent structures have been identified, we often are interested in the original properties of particles that are part of a particular group. For performance and memory optimization, the particles in `hop` are given only the attributes necessary to identify individual clumps, whereas the original particle set (`sph_particles` in the above example) may contain additional information, such as chemical composition. We therefore want to find the intersecting subset of particles with the same key among the original `sph_particles` and the particles in `hop`. This is done in the line

```
blob = [x.get_intersecting_subset_in(sph_particles) for x in hop.groups()]
```

Finally, we stop `hop` and return the appropriate subset of particles.

8.5.4.4 *Converting from Hydro Back to Stars*

In Section 8.5.2, we discussed how a separate code can be used to manage a head-on collision between a $3 M_{\odot}$ and a $1 M_{\odot}$ star, each evolved to an age of 150 Myr, continuing the evolution of the resulting merged star afterwards. There we adopted the output of the MMAMS (Gaburov et al. 2008; Lombardi & Sawyer Warren 2014) package to resolve the collision, but now we can adopt the methods discussed in the previous subsection, using SPH rather than MMAMS to determine the structure of the merger product.

As before, we first evolve the stars for 150 Myr, which is half of the primary's main sequence lifetime. Both stars are then converted to SPH realizations and relaxed (see Section 8.5.4.1). The resulting particle sets are combined, with the stellar surfaces just touching and zero relative velocity at infinity. We then use the SPH code `Fi` to resolve the hydrodynamics of the encounter. The free-fall timescale at this moment is just a few minutes, but to ensure that the gas distribution is in equilibrium, we continue the hydrodynamical calculation for 10^3 times longer. After the SPH calculation, we identify the merged star by using `hop` to search for the most massive blob (see Section 8.5.4.3), and then we strip away any low-density unbound splash material. The leftover merger product has a mass of $3.73 M_{\odot}$, very close to the MMAMS result for the same encounter (see Section 8.5.2).

After stripping away the envelope and relaxing the SPH system again, we put the star back into the Henyey stellar-evolution code. This latter step is a rather delicate process because the stellar-evolution code expects a stellar structure that is in thermal equilibrium. The hydrodynamics blob will be in dynamical equilibrium, but thermal equilibrium cannot be guaranteed. The procedure for converting the SPH star into a Henyey model is

```
from amuse.ext.sph_to_star import convert_SPH_to_stellar_model
new_stellar_model = convert_SPH_to_stellar_model(star_in_SPH)
```

This function call can be quite expensive in terms of computer time, and may even fail after an hour or so of crunching. If it succeeds, however, you have a good chance of being able to evolve the star using MESA or EVtwin. After this conversion, we

continue the evolution of the collision product to study its track in the Hertzsprung–Russell (H–R) diagram.

Figure 8.9 displays the same information for this SPH/hop merger as was presented in Figure 8.7 for the MMAMS merger (see Section 8.5.2). The red, blue, and green tracks show, respectively, the evolution of main-sequence stars of masses $3 M_{\odot}$, $4 M_{\odot}$, and $3.73 M_{\odot}$ (the value returned by SPH/hop). The yellow dashed line is the evolutionary track of the merged object. As with the merger discussed in Section 8.5.2, the evolutionary track of the merger product makes several rapid excursions in temperature and luminosity before settling (after $\sim 10^5$ yr, again comparable to the Kelvin–Helmholtz timescale; see Equation (5.6)) onto a new stable track. Its subsequent evolution takes it along a significantly more luminous track than either the MMAMS equivalent or the $4 M_{\odot}$ star.

Some cautionary notes are in order. First, recognizing the bound blob after the hydrodynamical simulation depends on several parameters in the hop algorithm (see Section 8.5.4.3), and may require some experimentation. Second, we have used a relatively small number of SPH particles (100 equal-mass particles per solar mass) here to model the merging stars. In part, this is to reduce the cost of the SPH simulation, but it also reflects the fact, already noted, that converting from an SPH to a Henyey representation is a very fragile process that may easily fail to produce a workable Henyey model, and whose outcome depends sensitively on the details of how SPH particles are combined into Henyey shells. In particular, the initial

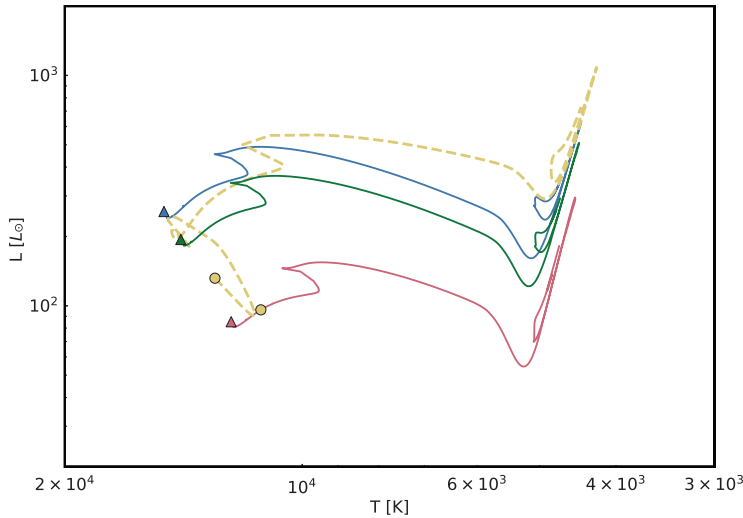


Figure 8.9. Evolutionary tracks for a $3 M_{\odot}$ star (red curve), a $4 M_{\odot}$ star (blue), and a $3.73 M_{\odot}$ star (green) from the main sequence (filled triangles) to the bottom of the AGB. The yellow dashed curve shows the evolutionary track of the merger product of a $3 M_{\odot}$ star and a $1 M_{\odot}$ star that collide halfway along the primary’s main-sequence track, with the merger structure determined using SPH and hop. The yellow circles represent the evolutionary states of the primary immediately before the collision and of the merger product immediately afterward. This figure was produced using the script `amuse.examples.merge_two_stars_and_evolve`, which should run in 10–15 minutes on a laptop.

location of the merger product on the H–R diagram can vary dramatically with these details, and probably should not be taken too seriously. As with many advanced topics in AMUSE, these details are still under active development: see the updates page `AMUSE_updates.pdf` for new material.

It is hard to validate the evolution of the merger product. Understanding the causes of these deviations from the standard evolutionary track would require a detailed analysis of the stellar structure, which could easily expand into a PhD project. We will not dwell further on the dark art of stellar astrophysics. It is clear, however, that it is worth exploring the possibility of resolving stellar collisions using hydrodynamical models. At this point, however, we consider MMAMS a viable option that at least leads to predictable behavior (see Section 8.5.2).

8.6 Complex Linear Code-coupling Strategies Applied

8.6.1 Heating of a Protoplanetary Disk

The abundance of several species of short-lived radionuclides (SLRs), such as ^{26}Al and ^{60}Fe , in the solar system has puzzled astronomers for some time (MacPherson et al. 1995). Key questions include: How did these volatile elements, particularly ^{26}Al , enter the solar system? How did they come to be part of meteorites? And, how did chondrules (the metallic grains containing these elements) form before chondrites (the meteorites that contain them)? There seems to be a lot we do not really understand.

One proposed explanation involves a nearby supernova when the Sun was still part of its parent star cluster. This is a two-stage process. First, the radiation from a bright supernova illuminates and heats the disk. Second, a few decades later, the hot ionized gas from the explosion plows through it. The idea is that it might be possible for the first burst of radiation to heat the disk enough to vaporize the dust, such that the subsequent wave of radioactive material could deposit the SLRs. Later cooling of the disk might subsequently have caused these SLRs to become part of the newly formed dust. This is all speculative, but worthy of further exploration.

Here, we study the first stage of the process: the heating and ionization of a protoplanetary circumstellar disk by the radiation from the supernova. For the gaseous disk, we adopt the protoplanetary disk model discussed in Section 6.4.1. To mimic the thick protoplanetary disk, we adopt a Safronov–Toomre Q parameter of $Q = 25$, rather than $Q = 2$ as in the earlier example of a thin disk (see Section 6.4.1). The disk was constructed as in the earlier example, with 10^4 equal-mass SPH particles at distances between 1 au and 100 au from the Sun, a total mass of $0.01 M_{\odot}$, and a power-law density profile with an exponent of -2. We arbitrarily assume that the disk is oriented at an angle of 15° (almost face-on) to the line of sight of the exploding star. The hydrodynamical simulation was run using `Fi` with an isothermal equation of state and an artificial viscosity parameter of $\alpha = 0.1$ (see Section 6.2.4).

For the evolution of the supernova luminosity, we follow Sanders et al. (2015) and adopt a simple broken power-law luminosity function with characteristic luminosity and time scales. The script `amuse.examples.supernova_iip_Lightcurve` presents code from Sanders et al. (2015) to reproduce those light

curves. We consider two distinct supernovae, with peak luminosities of a few times $\sim 10^9 L_{\odot}$, reached about 10 days after the onset of the explosion. For the first, we adopt the best-fit parameters for type-IIp supernova SN PS1-11aof, which has a peak luminosity of 2×10^{43} erg/s. The second supernova, PS1-10a, is somewhat dimmer. The light curves for the two supernovae are shown as the blue curves in Figure 8.10.

We used the radiative-transfer code SPHRay, which is well suited to such simulations, because it does not impose any limits on the spatial geometry or the computational domain. The hot supernova radiation was mimicked by setting some additional parameters in the radiative-transfer code.

```
radiative = SphRay(redirection="file", number_of_workers=24)
radiative.parameters.number_of_rays = number_of_rays / time_step
radiative.parameters.default_spectral_type = -3.0
```

Here, `radiative.parameters.number_of_rays = number_of_rays / time_step` defines the number of rays per time step used in the radiative-transfer

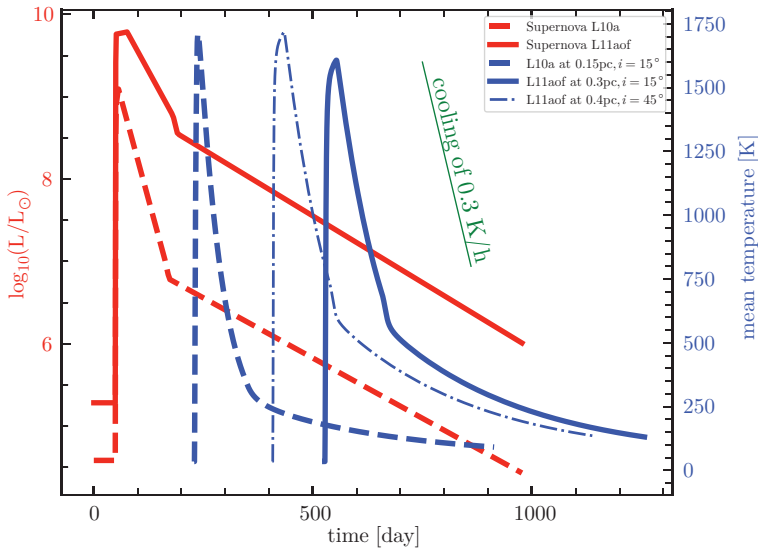


Figure 8.10. Light curves for the type-IIp supernovae PS1-10a and PS1-11aof (dashed and solid blue curves, respectively, scale in blue on the left, Sanders et al. 2015). The red curves give the temperature of our model disk at distances of 0.15, 0.3, and 0.4 pc (from left to right). The mean disk temperature (axis scale in red on the right) gives the disk temperature calculated using supernova PS1-10a at $d = 0.15$ pc and $\Theta = 15^\circ$ (the dashed curve) and PS1-11aof at a distance of 0.3 pc with $\Theta = 45^\circ$ and at $d = 0.4$ pc with $\Theta = 15^\circ$ (the solid curves). The thin green line (upper right) indicates a cooling of 0.3 K/hour, which is consistent with cooling after the supernova radiation has heated the disk for SN PS1-11aof, and which is sufficient for the formation of vitreous structures in the chondrules of the circumstellar disk. The calculations are performed at a resolution in the radiative-transfer code of $N_{\text{ray}}/dt=10^7/(1\text{units.hour})$. The figure was made using the script `amuse.examples.supernova_irradiation`. The calculation took several days on a laptop, and for this reason we include the data files and the routine to plot those in `amuse.examples.plot_supernova_IIP_and_disk_temperature`.

solver. In our case, we adopted `number_of_rays / time_step=107/(11 units.hour)`. The parameter `default_spectral_type` is used to indicate the slope of the source's spectral energy distribution. For a supernova, a value of -3 is closer to the actual observed spectral energy distribution than the default value of -1 .

In the radiation transport code, the irradiating source is represented by a particle, called `src_particle`. We can update this particle's parameters at run time by using a separate channel.

```
channel_from_src_to_rad = \
    supernova.particles.new_channel_to(radiative.src_particles)
supernova.evolve_model(time)
channel_from_src_to_rad.copy_attributes(["luminosity"])
```

Note that there is only one source particle in our calculation, even though the code's denotative is plural.

The calling sequence for the radiative-transfer and hydrodynamics modules is similar in structure to Code example 8.2 in Section 8.2.1, where we combined stellar evolution with gravitational dynamics. The two codes are called `radiative` and `hydro`. Both codes have appropriate channels defined to and from the local particle sets: `disk` for the SPH particles, `star` for the stellar particle in the hydro code, and `supernova.particles` for the local source particle to the radiative-transfer code. The event loop then becomes:

```
while time < time_end:
    time += time_step / 2
    supernova.evolve_model(time)
    channel_from_src_to_rad.copy_attributes(["luminosity"])
    radiative.evolve_model(time)
    channel_rad_to_hydro.copy()

    time += time_step / 2
    hydro.evolve_model(time)
    hydro_to_disk.copy()

    supernova.evolve_model(time)
    channel_from_src_to_rad.copy_attributes(["luminosity"])
    update_source_particle(time)
    radiative.evolve_model(time)
    rad_to_disk.copy()
```

First, we update the supernova luminosity, perform an evolution step with a time step of `time_step / 2` in the radiative-transfer code, and copy the irradiated SPH particle data to the hydrodynamics code. Next, we update the time to a full step, evolve the hydrodynamics of the disk, and copy the data back to the framework. Finally, we repeat the first snippet, updating the supernova luminosity and performing another half radiative time step. In the last line, we copy the data from the radiative-transfer code back to the local particle set.

The red curves in Figure 8.10 show the temperature evolution of three identical protoplanetary disks around a $1 M_{\odot}$ star at a distance of 0.15 pc from SN PS1-10a (left dashed curve), and at 0.3 and 0.4 pc from the considerably brighter SN PS1-

11aof (right two solid curves). The mean temperature in the disk exceeds 1500 K for each of these supernovae and distances, which is high enough to melt agglomerates with the highest melting points. Carrying out any of these simulations with a lower resolution, using less than 10^7 rays per hour, in the radiative transport calculation would lead to lower temperatures, whereas adopting a larger number of rays hardly affect the results, except that the script takes longer to run.

The shock wave reaches the disk a few tens of years after supernova radiation. This is a more hydrodynamical process, which we will discuss further in Section 10.3.

8.6.2 Integrating a Star Cluster with Interrupts

The close proximity of the Trapezium cluster allows detailed observations of circumstellar disk sizes using HST/WFPC2 (Vicente & Alves 2005). This size distribution is well described by a power law, but the origin of this distribution is not well understood. Vicente & Alves (2005) argue that “[Despite] the young age of the Trapezium, given that disk destruction is well underway, it is perhaps too late to tell if the present-day disk size distribution is primordial or if it is a consequence of the massive star formation environment.”

We can write a small AMUSE script to test this hypothesis, and to study the evolution of the size distribution of circumstellar disks. Such a script should integrate the equations of motions of the stars and keep track of close encounters in order to truncate disks depending on the distance, mass, and velocity of a close encounter. Any direct N -body code will be sufficiently accurate for such a study. And since the number of stars is relatively small, these calculations can be performed on a laptop. Since the vast majority of close encounters are simple two-body interactions, we can integrate the cluster for a small time frame (smaller than a typical crossing time of ~ 2.4 Myr, see Equation (4.5)), evaluate all close encounters (using the `Kepler` package to find the distances of closest approach), resolve the effect of the encounters on the circumstellar disks, and compare the numerically obtained distribution of disk sizes with the observed distribution.

Code examples 8.14 to 8.16 present the main routine for these runs. The complete script is available in `amuse.examples.gravity_kepler_disks`. It generates the data for Figure 8.11, and should take 5–10 minutes to run on a laptop. This simulation started with 2000 stars drawn from a Kroupa (2001) distribution with masses between $0.01 M_{\odot}$ and $100 M_{\odot}$, with positions drawn from a fractal distribution (Goodwin & Whitworth 2004) of dimension $F = 1.6$, and velocities scaled so that the cluster was in overall virial equilibrium. At run time, we check for close encounters. If such an event is identified, we calculate the pericenter distance for the two-body encounter and truncate the disks of both stars.

The disk truncation radius, r_{trunc} , is calculated from the distance of closest approach between two stars, q , and the masses of the two stars m and M by (Breslau et al. 2014)

$$r_{\text{trunc}} = 0.28q \left(\frac{m}{M} \right)^{0.32}. \quad (8.1)$$

```

97     while time < t_end:
98         time += dt
99
100        gravity.evolve_model(time)
101        Etot_prev_se = gravity.kinetic_energy + gravity.potential_energy
102        while stopping_condition.is_set():
103            channel_from_gd_to_framework.copy()
104            Ek_enc = gravity.kinetic_energy
105            Ep_enc = gravity.potential_energy
106            for ci in range(len(stopping_condition.particles(0))):
107                particles_in_encounter = Particles(
108                    particles=[stopping_condition.particles(0)[ci],
109                             stopping_condition.particles(1)[ci]])
110                local_particles_in_encounter = particles_in_encounter.
111                    get_intersecting_subset_in(bodies)
112
113                resolve_close_encounter(gravity.model_time,
114                                        local_particles_in_encounter, Johannes)
115
116                Nenc+=1
117                print (f"At time= {gravity.model_time} Nenc= {Nenc} ",
118                      (f"Rdisk= {local_particles_in_encounter.disk_radius}")
119                      channel_from_framework_to_gd.copy_attributes(["radius"]))
120                dEk_enc += Ek_enc - gravity.kinetic_energy
121                dEp_enc += Ep_enc - gravity.potential_energy
122                gravity.evolve_model(time)
123
124        channel_from_framework_to_gd.copy_attributes(["mass"])

```

Code example 8.14: Main routine of a script to perform an N -body simulation with interrupts to resolve star-disk encounters. The main event loop is presented in Code example 8.15. The star-disk interaction routine is given in Code example 8.16.

```

8 def resolve_close_encounter(time, bodies, Johannes):
9     Johannes.initialize_from_particles(bodies)
10     rcom = bodies.center_of_mass()
11     vcom = bodies.center_of_mass_velocity()
12
13     a, e = Johannes.get_elements()
14     p = Johannes.get_periastron()
15     print("Close encounter at t=", time.in_(units.Myr), "a=", a.in_(units.AU), "e="
16           , e, "p=", p.in_(units.AU), "M=", bodies.mass.max().in_(units.MSun), bodies.mass
17           .min().in_(units.MSun), "at d=", rcom.in_(units.parsec), "with v=", vcom.in_(
18             units.kms))
19
20     truncate_disks_due_to_encounter(bodies, p)

```

Code example 8.15: Continuation of the script to run an N -body simulation with interrupts to resolve star-disk encounters. The star-disk interaction routine is given in Code example 8.16.

If r_{trunc} is smaller than the current disk size, the disk is truncated at r_{trunc} . The truncation routine is presented in Code example 8.16,

Figure 8.11 compares the size distributions of the circumstellar disks with the observed distribution in the Orion Trapezium cluster. The observed distribution is

```

21 def truncate_disks_due_to_encounter(bodies, p):
22     q = bodies[1].mass/bodies[0].mass
23     rtr_prim = 0.28*p / q**(0.32)
24     rtr_sec = 0.28*p * q**(0.32)
25     dm0 = truncate_disk_due_to_encounter(bodies[0], rtr_prim)
26     dm1 = truncate_disk_due_to_encounter(bodies[1], rtr_sec)
27
28     mtot = bodies.mass.sum()
29     bodies[0].accreted_mass += dm1 * bodies[0].mass/mtot
30     bodies[1].accreted_mass += dm0 * bodies[1].mass/mtot
31
32     bodies[0].radius = min(bodies[0].radius, 0.5*p)
33     bodies[1].radius = min(bodies[1].radius, 0.5*p)
34
35     bodies[0].mass = bodies[0].stellar_mass + \
36         bodies[0].disk_mass + bodies[0].accreted_mass
37     bodies[1].mass = bodies[1].stellar_mass + \
38         bodies[1].disk_mass + bodies[1].accreted_mass

```

Code example 8.16: Routine to truncate a circumstellar disk during a pericenter passage with another star.

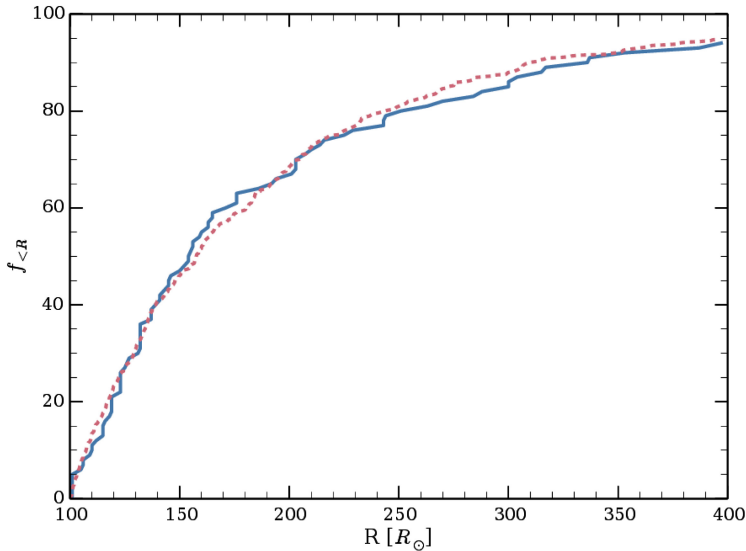


Figure 8.11. Cumulative size distribution of circumstellar disks in the Trapezium cluster. The observations are complete for disk radii exceeding 100 AU. The red dotted line gives the 95 observed disk sizes from the observations. The black solid curve gives the result of our simulations at 1 Myr. A Kolmogorov–Smirnov (KS) comparison (Kolmogorov 1933; Smirnov 1948) of the two cumulative distributions results in a KS-statistic of 0.058 with a two-tailed p-value of 0.94, indicating that we cannot reject the hypothesis that both distributions are indistinguishable. The complete script that produced this figure is in `amuse.examples.compare_disk_size_Trapezium`. It simply plots data read from a file and runs in a few seconds on a laptop.

limited by the telescope’s resolution (Vicente & Alves 2005). The pixel size in these observations is 45 AU, and disks as small as 1.5 pixels (or 67.5 AU) can be resolved. The authors argue that their sample is complete down to a minimum radius of

100–150 AU. Of the 162 protoplanetary disks, 95 are larger than 100 AU (Vicente & Alves 2005). Although the observations are limited and the dynamical model is oversimplified, the similarity between the two datasets is striking.

8.7 Multiscale Modules in AMUSE

8.7.1 The Multiples Module: Combining Few-body with Many-body Dynamics

The N -body integrators described in Chapter 4 are designed to move large numbers of stars around a stellar system, and they are extremely efficient in accomplishing that task. However, in reality many stars are born with a companion, in tightly bound binary systems, or they may acquire companions through dynamical interactions. In these circumstances, while many of the direct N -body codes in AMUSE can still follow the motion with reasonable accuracy, the short time steps associated with the tight orbits mean that the integration effectively grinds to a halt once close binaries form.

In traditional N -body codes, such as NBODY6/7 and `starlab`, a substantial fraction of the total code base is devoted to following the internal motion of binaries and multiples simultaneously with the large-scale motion of the rest of the system. The algorithms involved have varying degrees of efficiency and accuracy, but all gradually relax the accuracy of the perturbed binary integration until “unperturbed” binaries (usually at a perturbation level of 1 part in 10^{6-7}) are treated as isolated unperturbed objects. The alternative approach, widely used in Monte Carlo schemes, treats multiples (where for simplicity the term will henceforth include binaries) as completely unperturbed until they interact strongly with another object. (A pioneering N -body variant on this approach was the GORILLA code presented by Tanikawa & Fukushige 2009.) Many papers have been devoted to the relative merits of these approaches, but, in our opinion, there is no convincing evidence that the improved treatment of lightly perturbed binaries inherent in the N -body approach has any significant advantage over the much simpler Monte Carlo treatment.

Accordingly, in the AMUSE multiples module, we depart from the N -body norm and adopt the Monte Carlo approach to multiple interactions. The module is designed to handle close encounters between stars and multiples in N -body systems. The basic idea is that the N -body modules manage multiple centers of mass, but are completely ignorant of the internal dynamics of these systems. In operation, the multiples module maintains a database of all stable multiples in the system. A stable multiple is a binary, a hierarchical triple deemed stable by the Mardling (2008) criterion, or a higher-order multiple system in which the Mardling criterion applied to the outermost orbits indicates approximate stability. As illustrated in Figure 8.12, when two centers of mass approach within the sum of their (dynamical) radii, a stopping condition is thrown and control is passed to multiples. The radii are typically chosen to be the 90° turnaround distance ($\sim G\langle m \rangle / \langle v^2 \rangle$) for a single star, and a few outer binary semimajor axes for a multiple system. In virial equilibrium and N -body units, this distance is $1/2N$. (All critical multiples parameters have reasonable defaults and can be modified by the user script, if desired.)

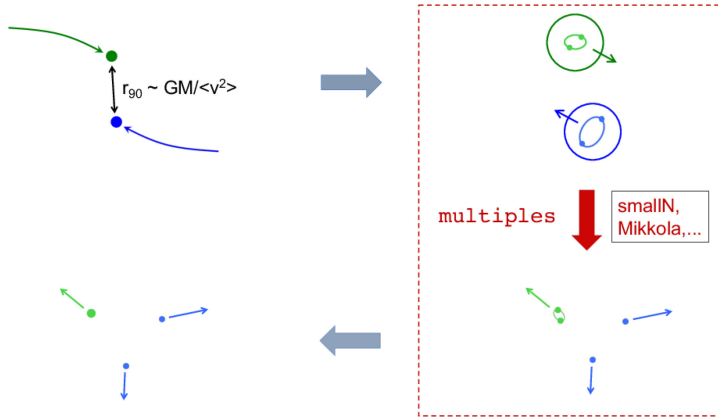


Figure 8.12. Two interacting stars (top left) are identified by the N -body code, expanded into their component stars (top right), integrated to completion by a standalone small- N code (bottom right), and reinstated into the N -body code (bottom left).

Once a dynamical interaction is detected and passes some basic tests (e.g. that the centers of mass are actually approaching), the internal structures of the interacting bodies are reinstated from the `multiples` database and the dynamics of the resulting small- N system followed to completion using a dedicated code. The AMUSE `smallN` module, a robust shared-step time-symmetrized code with analytic extensions (Hut et al. 1995; McMillan & Hut 1996) is often used for this task; the Mikkola module, which implements Mikkola’s (Mikkola & Merritt 2008) algorithmic regularization code is valuable for problems where post-Newtonian gravitational effects must be included. “Completion” in this context means that the system has resolved itself into a collection of mutually receding stable systems (as just defined). We always expect this to be the eventual outcome of a small- N interaction. In principle, the tidal field of the N -body system could be included in the small- N calculation, but this is not currently implemented, although an attempt is made to absorb tidal errors by reorienting the final top-level system. To enforce a degree of isolation, in the current implementation a close perturber (lying within ~ 3 times the initial separation of the interacting centers of mass) will veto the small- N calculation.

A major approximation in this approach is that interactions occur instantaneously—two center-of-mass particles in the N -body code are replaced by some number of new center-of-mass particles with no change in the system time. For most interactions, which take only a two-body encounter time ($\sim 1/N$ dynamical times) to complete, this causes a negligible error, but in some long-lived “hierarchical” resonances, as defined by Hut (1993), this may not be acceptable. We address this shortcoming as follows. In a pure scattering experiment, we would require that all top-level centers of mass of stable multiples be mutually receding and unbound at the end of the simulation. However, if we also accept centers of mass that form wide bound pairs, hierarchical resonances resulting in long bound intermediate orbits can be returned to the N -body code, even though the criteria for completion are not fully met. The internal motion is frozen until the N -body code again selects these bodies to continue the interaction.

Thus, long resonances are not treated as instantaneous, providing a rudimentary means for the N -body code to interact with the small- N system.

Allowing wide bound systems as an acceptable outcome of a scattering has another important consequence. In practice, it is very convenient to use `multiples` to also manage two-body encounters of single stars, even though most could easily be managed by the N -body integrator. This allows `multiples` to detect wide binaries formed by the N -body code, allowing us to model binary formation as well as binary interactions.

Code example 8.17 illustrates a simple `multiples` integration. Basically, we set up the system and tell `multiples` what N -body, `kepler`, and small- N codes to use and (since `multiples` is written in N -body units) optionally specify a converter.

```

55 def integrate_system(number_of_stars, t_end, seed=None):
56
57     total_mass = number_of_stars | units.MSun
58     length = 1 | units.parsec
59     converter = nbody_system.nbody_to_si(total_mass, length)
60     gravity = ph4(convert_nbody=converter)
61     gravity.initialize_code()
62     gravity.parameters.set_defaults()
63     gravity.parameters.epsilon_squared = (0.0 | units.parsec)**2
64
65     if seed is not None:
66         numpy.random.seed(seed)
67     stars = new_plummer_model(
68         number_of_stars, convert_nbody=converter)
69     stars.mass = total_mass / number_of_stars
70     stars.scale_to_standard(
71         convert_nbody=converter,
72         smoothing_length_squared=gravity.parameters.epsilon_squared
73     )
74     index = numpy.arange(number_of_stars)
75     stars.id = index+1
76     stars.radius = (0.5 / number_of_stars) | units.parsec
77
78     gravity.particles.add_particles(stars)
79
80     stopping_condition = gravity.stopping_conditions.collision_detection
81     stopping_condition.enable()
82
83     init_smalln(converter)
84     kep = Kepler(unit_converter=converter)
85     kep.initialize_code()
86     multiples_code = multiples.Multiples(gravity, new_smalln, kep,
87                                         constants.G)
88     multiples_code.neighbor_perturbation_limit = 0.05
89     multiples_code.global_debug = 1
90
91     energy_initial = print_diagnostics(multiples_code)
92     multiples_code.evolve_model(t_end)
93     print_diagnostics(multiples_code, energy_initial)

```

Code example 8.17: A simple N -body solver with close-encounter management. The full script can be found in `amuse.examples.basic_multiples`. It should run in a few seconds on a laptop.

```

13 SMALLN = None
14 def init_smalln(converter):
15     global SMALLN
16     SMALLN = SmallN(convert_nbody=converter)
17 def new_smalln():
18     SMALLN.reset()
19     return SMALLN

```

Code example 8.18: Initializing the small- N part of the multiples solver.

From that point on, we use multiples in exactly the same way as any other integrator—its `evolve_model` method invokes the integrator and takes appropriate action if encounters occur. The full script, including diagnostic and other functionality, can be found in `amuse.examples.basic_multiples`.

Note that the stellar ID defined in the script is important because it is used for internal bookkeeping within the module. The only slightly unusual aspect of the multiples setup is that, instead of providing the small- N module as an argument, the current module expects a function that resets and returns a small- N integrator (see Code example 8.18). This was part of the original design of the module, and is currently under review; it may be eliminated in future versions of AMUSE.

Finally, the multiples database enables the implementation of treatments of binary evolution (Section 5.3), secular internal evolution of multiple structure (Section 4.4.4), e.g., by the von Zeipel–Lidov–Kozai (von Zeipel 1910; Lidov 1962; Kozai 1962) mechanism, and external perturbations due to wide encounters in the N -body code. These are not yet fully implemented, and represent work in progress.

8.7.2 The Kira Module: Combining Stellar and Binary Evolution with Stellar Dynamics

In 1992, with Piet Hut and Jun Makino, we began work on the `starlab` project (Portegies Zwart et al. 1999). Central to this effort were two codes, the stellar-evolution code `SeBa` and a fourth-order accelerated Hermite integrator not dissimilar to `ph4`. The combined code was known as `kira`. Many of the fundamental ideas we learned in creating `kira` have percolated into the AMUSE framework.

One aspect in which `kira` was ahead of AMUSE, as described thus far, was in its handling of binary evolution in conjunction with the dynamics. So far, we have briefly discussed the evolution of binary stars in Section 5.3, but that was for binaries in isolation and not in a dynamical environment. Here, we adopt the same fundamental structure for single stars and binary stars to simulate a cluster with primordial binaries. Binaries and higher-order multiple encounters are managed by the encounter module in `amuse`, and it is possible to ionize binaries or make new binaries by dynamical encounters. We call this script `kira.py` in recognition of the original package. It is available in `amuse.examples.kira`. We use the `multiples` module to treat close multiple stellar encounters (see Section 8.7.1). Like many of the more complex AMUSE scripts described in this book, this is a work in progress and likely to change.

There are a few additional details to consider when modeling a star cluster containing binaries (either primordial or dynamically formed). The small orbital periods of binaries tend to dramatically slow down the underlying N -body code, and the many small time steps required to resolve the binaries lead to a steady accumulation of round-off and integration errors. These factors mandate special methods to manage the binaries. A further complication is that binaries are affected both by the stellar evolution of the components and by occasional encounters with other stars (and binaries) in the system. The evolution of single stars can be handled by any of the stellar-evolution codes available in AMUSE. Binary evolution, however, is a different matter.

The initial conditions for the simple simulation described here begin with a Plummer sphere of 100 particles with masses randomly selected from a Salpeter mass function between $0.1 M_{\odot}$ and $100 M_{\odot}$. We then select $N_{\text{bin}}=50$ stars randomly using

```
binaries = center_of_masses.random_sample(Nbin)
```

These stars are then provided with a secondary companion, with orbital parameters chosen from standard distributions. The mass of the secondary is selected randomly from a uniform distribution between the minimum stellar mass and that of the primary. The orbital eccentricity is selected between 0 and 1, assuming a thermal distribution ($p(e) = 2e$; Duquennoy & Mayor 1991), and the semimajor axis is selected using the prescription in Equation 1.6.3 of Eggleton (2006), with a period between 10 days and 100 years. Other binary orbital parameters are initialized in much the same way as described in Section 5.3.

With stars and binaries initialized, we can start the stellar-evolution code and the global gravity solver. We use *SeBa* to resolve the stellar evolution because it has a natural way to handle the evolution of single stars as well as binaries. For the global dynamical evolution of the stars, we use *Hermite*. Stable binaries and close two-body encounters are not integrated, but we follow their dynamical evolution via the *Kepler* package. The global N -body code integrates the center of mass of the close two-body encounters as well as of the center of masses of the stable binary systems. Once another star perturbs a binary (or higher-order system), we resolve its interaction using the *multiples* module (Section 8.7.1). The *smallN* code it relies on is based on the old *Kira_smallN* package from *starlab*.

We start by importing the *encounters* module, which is a general-purpose module designed to handle stellar encounters of all types (collisions, multiple encounters, etc.) and is still under development

```
from amuse.couple import encounters
```

We then realize *encounters_code* using the *kepler* package to handle stable two-body systems, and *smallN* for multibody systems

```
encounter_code = encounters.HandleEncounter(
    kepler_code = new_kepler(converter),
    resolve_collision_code = new_smalln(converter)
)
```

Here, we use helper functions `new_kepler` and `new_smallN`, which return instantiations of the two codes with common parameters (such as `timestep_`-parameter in `smallN`) set to reasonable values. We now start `multiples` handling using `Multiples`, which is part of the `encounters` module. We give it a global gravity code and the `encounter_code` we just created using the `kepler` and `smallN` modules:

```

multiples_code = encounters.Multiples(
    gravity_code = Hermite(converter),
    handle_encounter_code = encounter_code
)

```

Multiple encounters will now be resolved using `smallN`, whereas two-body stable binaries are addressed using the `kepler` solver, as described in Section 8.7.1.

Having set up all code dependencies, it is time to copy all stellar and binary data to the `multiples` module and declare channels to the code:

```

multiples_code.particles.add_particles((stars+binary_stars).copy())
multiples_code.singles_in_binaries.add_particles(singles_in_binaries)
multiples_code.binaries.add_particles(binary_stars)
multiples_code.commit_particles()
channel_from_stars_to_particles = (
    stellar.particles.new_channel_to(multiples_code.particles)
)

```

The `multiples` module now takes care of all two-body and higher-order interactions in the star cluster simulation.

After an encounter, binary parameters may have changed. This affects the stellar and binary evolution of these systems, and the stellar-evolution code must be informed of these changes. We do this via a stopping condition, in this case `binaries_change_detection`, which stops the `multiples` module when the orbital separation or eccentricity of a binary is significantly affected by the dynamical encounter:

```

stopping_condition \
    = multiples_code.stopping_conditions.binaries_change_detection
stopping_condition.enable()

```

We run the `multiples` module in a (by now) familiar loop:

```

while time < time_end:
    time += time_step
    multiples_code.evolve_model(time)

    if stopping_condition.is_set():
        resolve_changed_binaries(stopping_condition, stellar)

    stellar.evolve_model(ti)
    channel_from_stars_to_particles.copy_attributes(["mass", "radius"])
    update_dynamical_binaries_from_stellar()

```

Newly formed binaries, binaries with orbital elements affected by a dynamical encounter, and binaries that broke up (and must be removed) are handled by the code in Code example 8.19. (Note that we encountered the routine `calculate_orbital_elements` previously in Section 4.4.1.)

Similarly, after the stellar-evolution step we must update the dynamical information in the binaries. This is realized by reading the binary parameters in the stellar-evolution code, calculating the new coordinates of the binary components, and updating the dynamics. Special care is required for binaries that merged during the evolution process, or that were disrupted in a supernova explosion, but those can be handled with separate stopping conditions for `supernova_detection` and `merger_detection` in the stellar-evolution module.

```
def resolve_supernova(stellar, stars, multiples_code):
    singles = stars.as_particle_in_set(multiples_code.singles)
    singles_in_stellar = singles.as_particle_in_set(stars)
    singles.velocity += singles_in_stellar.supernova_kick_velocity
```

Figure 8.13 presents the results of a small test simulation using the script `kira` from the `amuse.examples` module. The initial conditions for these calculations are a standard star cluster with 100 stars drawn from a Salpeter (1955) mass function with a minimum mass of $10 M_{\odot}$, distributed in a Plummer (1911) sphere with a characteristic radius of 1 pc. Half of the primary stars are given a secondary companion

```
134
135
136 def resolve_changed_binaries(stopping_condition, stellar, converter):
137     new_binaries = stopping_condition.particles(0)
138     for bi in new_binaries:
139         print(("add new binary:", bi))
140         a, e = calculate_orbital_elementss(bi, converter)
141         bi.semi_major_axis = a
142         bi.eccentricity = e
143         stellar.binaries.add_particle(bi)
144         print(("new binary parameters", a, e))
145         print(bi)
146
147     lost_binaries = stopping_condition.particles(1)
148     for bi in lost_binaries:
149         print(("remove old binary:", bi.key))
150         stellar.binaries.remove_particle(bi)
151
152     changed_binaries = stopping_condition.particles(2)
153     for bi in changed_binaries:
154         bs = bi.as_particle_in_set(stellar.binaries)
155         a, e = calculate_orbital_elementss(bi, converter)
156         bs.semi_major_axis = a
157         bs.eccentricity = e
158         print(("Modified binary parameters", a, e))
159         print(bs)
```

Code example 8.19: Morphing binaries in `kira.py`.

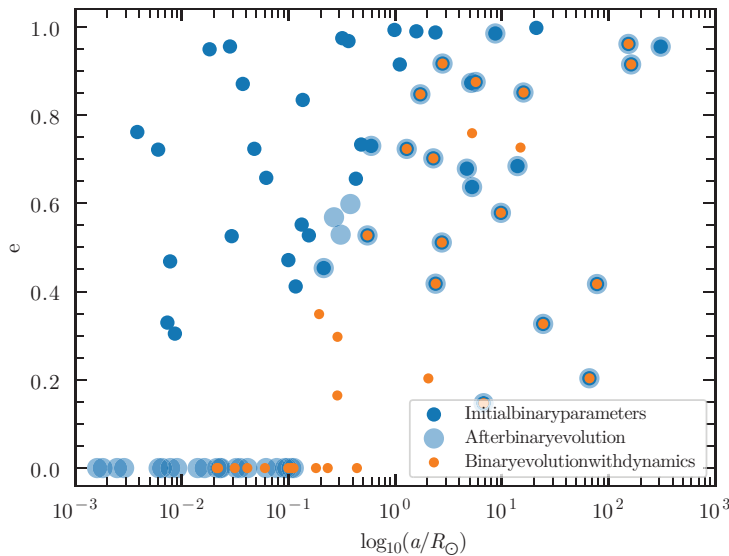


Figure 8.13. Distribution in semimajor axis and eccentricity of a population of binary stars in an N -body system at $t = 0$ Myr (dark blue bullets) and at an age of 10 Myr (big blue and orange bullets). The big blue bullets indicate what happens in stellar evolution but without taking the dynamical evolution into account. The orange bullets show the same binary population but they are affected by stellar evolution, binary evolution, and by stellar dynamics. We deliberately plot the blue dots first and larger than the red dots to illustrate how some of the binaries disappear or move within the parameter space. The script that produced this figure is `amuse.examples.kira`. It should run in about a minute on a laptop.

with mass chosen randomly between $10 M_{\odot}$ and the primary mass, in an orbit with eccentricity taken from a thermal distribution and semimajor axis drawn from the distribution proposed by Eggleton (2006). The other binary parameters and the binaries’ orientations in space are random. We integrate the equations of motion using the Hermite integrator—see Section 4.3.2; binary evolution is managed using `SeBa` (Portegies Zwart & Verbunt 1996; Portegies Zwart & Yungelson 1998; Toonen et al. 2012; Portegies Zwart & Verbunt 2012).

Figure 8.13 shows the distributions of orbital semimajor axis and eccentricity of the binaries after integrating for 10 Myr. We integrate for such a short time because that is already sufficient to see differences compared to a calculation without stellar and binary evolution.

The blue bullets in Figure 8.13 indicate the initial conditions (orbital separation and eccentricity) of the primordial binaries in the simulation. The red dots show the orbital parameters after 10 Myr of integration using `kira.py`. For several of the closest binaries the orbital parameters have already been affected by stellar evolution and tidal interactions within the binary. Most of these are affected by tidal circularization, particularly those with short orbital periods and eccentricities $\gtrsim 0.6$. Some of the tightest ($a \lesssim 0.1 R_{\odot}$) circular binaries have disappeared because they were Roche-lobe overflowing at birth and have merged to form a single star.

8.7.3 The LonelyPlanets Module: Combining Planetary with Stellar Dynamics

Instead of integrating multiple stars in a cluster, one might be interested in the multiscale problem of integrating planetary systems. Integrating a single isolated planetary system is usually done with a regular N -body code, but a planetary system in a star cluster is quite challenging because of the wide range of time scales involved. One way to perform such a calculation is with a divide-and-conquer strategy.

For this purpose, we developed `LonelyPlanets` (not to be confused with the popular travel guide books, which deal with one specific rather peculiar planet). We mean to integrate multiplanet systems, possibly with moons, asteroids, and comets together with the dynamical evolution of the stellar cluster.

`LonelyPlanets` works through a two-stage process. In a first simulation, we select a star (in our example randomly), and monitor its most prominent (i.e. strongest) perturbers during the integration of the cluster. The event loop for `LonelyPlanets` is listed in Code example 8.20, and should look quite familiar by now, with the exception that we keep track of the perturbers. In this example, the perturber is any of the five nearest stars. In practice, one probably wants to keep track of the strongest perturber, rather than the nearest star, since a massive far away star can perturb a planetary system more severely than a low mass nearby star.

We find the nearest neighbors simply by sorting stars on the distance to our selected star,

```
def nearest_neighbors(bodies, star, Nnn=3):
    r = ((bodies-star).position-star.position).lengths().value_in(units.pc)
    r, p = (list(t) for t in zip(*sorted(zip(r, bodies-star))))
    b = Particles()
    for pi in p[:Nnn]:
        b.add_particle(pi)
    return b
```

```
173 while gravity.model_time < time_end:
174     gravity.evolve_model(gravity.model_time + dt)
175     channel_from_gd.copy()
176
177     star.age = gravity.model_time
178
179     target_star.append(star.copy())
180     model_time.append(gravity.model_time)
181     perturbers = find_perturbers(star, bodies, Nnn)
182     nn_distance[str(model_time[-1])] = (
183         star.position - perturbers.position
184     ).lengths()
185     nn_bodies.append(perturbers)
186     print(f"time={model_time[-1].in_(units.Myr)}, x={star.x.in_(units.pc)}")
187
```

Code example 8.20: Snippet of the stage 1 `LonelyPlanets` event loop.

In the second stage, we generate a planetary system around the selected star and integrate it together with the earlier (in stage 1) generated database of perturbers. We further optimize the stage 2 calculation (as illustrated in Code example 8.21) by including the perturbing stars only if their perturbations on the planetary system exceed some predetermined limit. In our case, we adopt a simple distance criterion of 3000 au. This saves a little computer time, but with only five perturbers and eight planets the gain is rather small.

```

286     for ti in range(len(model_time)):
287         star = planetary_system[planetary_system.type == "star"][0]
288         planetary_system.position -= star.position
289         planetary_system.velocity -= star.velocity
290         planetary_system.position += ss_star[ti].position
291         planetary_system.velocity += ss_star[ti].velocity
292         channel_to_gd.copy()
293
294         perturbed = nearest_encounter(
295             ss_star[ti], nn_bodies[ti], maximum_perturber_distance)
296         if perturbed:
297             npert += 1
298             print(f"Integrate with stars: N={len(nn_bodies[ti])} "
299                   f"at time={model_time[ti].value_in(units.Myr)}")
300             gravity.particles.add_particles(nn_bodies[ti])
301         else:
302             nunpert += 1
303         print(f"Time={model_time[ti].in_(units.Myr)}: "
304               f"N= {len(gravity.particles)} = "
305               f"{len(planetary_system)}+{len(nn_bodies[ti])}")
306         gravity.evolve_model(model_time[ti])
307
308         if perturbed:
309             gravity.particles.remove_particles(nn_bodies[ti])
310             channel_from_gd.copy()
311
312         star = planetary_system[planetary_system.type == "star"][0]
313         planets = planetary_system - star
314         get_orbital_elements_of_planetary_system(star, planets)
315         amin = (np.min(planets.semimajor_axis.value_in(units.au)) | units.au,)
316         sma.append(planets.semimajor_axis)
317         ecc.append(planets.eccentricity)
318
319         lost_planets = planets[planets.eccentricity > 1]
320         if len(lost_planets) > 0:
321             lp = lost_planets.get_intersecting_subset_in(gravity.particles)
322             gravity.particles.remove_particles(lp)
323         lost_planets = planets[np.isnan(planets.eccentricity)]
324         if len(lost_planets) > 0:
325             lp = lost_planets.get_intersecting_subset_in(gravity.particles)
326             gravity.particles.remove_particles(lp)
327

```

Code example 8.21: Stage 2 LonelyPlanets event loop, in which the planetary system is being integrated with nearby stars.

This is a typical example of optimizing for a rather minor speed-up while taking the risk of creating a complex bug. Fixing the bug may take more time than just integrating for all neighboring stars.

In Figure 8.14, we present the result of the stage 1 calculation of `LonleyPlanets`, where we initialize 1000 stars from a Kroupa mass function (between $0.08 M_{\odot}$ and $30 M_{\odot}$) in a virialized King model (with $W_0 = 5$) 1 pc radius. We randomly selected one star, which happened to have a mass of $\sim 0.11 M_{\odot}$ and was born in the cluster central region (at $(-0.026, -0.072, -0.14)$ pc from the center). This star acquired planets between 1 au and 100 au in circular orbits in the x - y -plane.

In the second stage of the calculation, we integrate the planetary system together with the most strongly perturbing stars. We select the three nearest neighbors and the three strongest perturbers. Sometimes these are the same stars, but sometimes a low-mass star happens to be near the planetary system, while other, more massive, stars are stronger perturbers. In Figure 8.15, we present the orbital separation and eccentricity of the 15 planets for the first 3 Myr of the evolution of the cluster. Although it is something of a jumble of lines, it is clear that the planet eccentricities

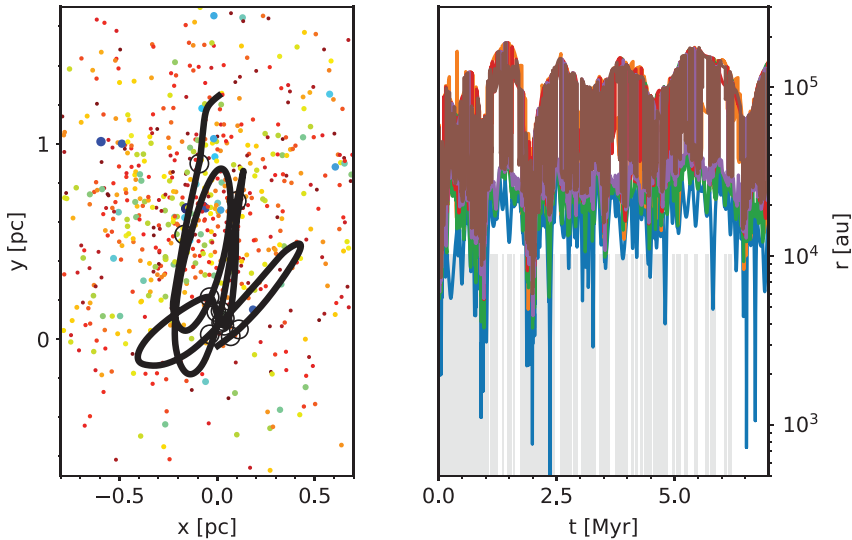


Figure 8.14. First stage calculation for `LonelyPlanets`. Strategy to reduce computing cost without loss of accuracy. This is a two-stage process. In the first step (left), we calculate the evolution of the cluster, storing all the close-encounter information. The points indicate the initial stellar positions (color and size represent mass). The orbit of a randomly selected star is presented as the solid black curve. The right panel shows, over the 3 Myr of the evolution in this cluster, the distance to the three nearest stars from the selected star. In the second step of the calculation, we repeat the close encounters of our selected star with the nearest other stars, but then including planets and asteroids. Left: A representation of the cluster at zero age, with the selected star’s orbit presented as a solid black curve. While integrating, we keep track of the strongest perturbers, and a small circle is drawn each time such a perturbation exceeds the equivalent of an encounter at around 5000 au. Right: The distance from our selected stars to the three strongest perturbers, as a function of time. The identity of these perturbers changes regularly, once their perturbation is overtaken by that of another (more strongly perturbing) star. The vertical dotted lines indicate the moments of strongest perturbation (and identify to the circles in the left-hand panel).

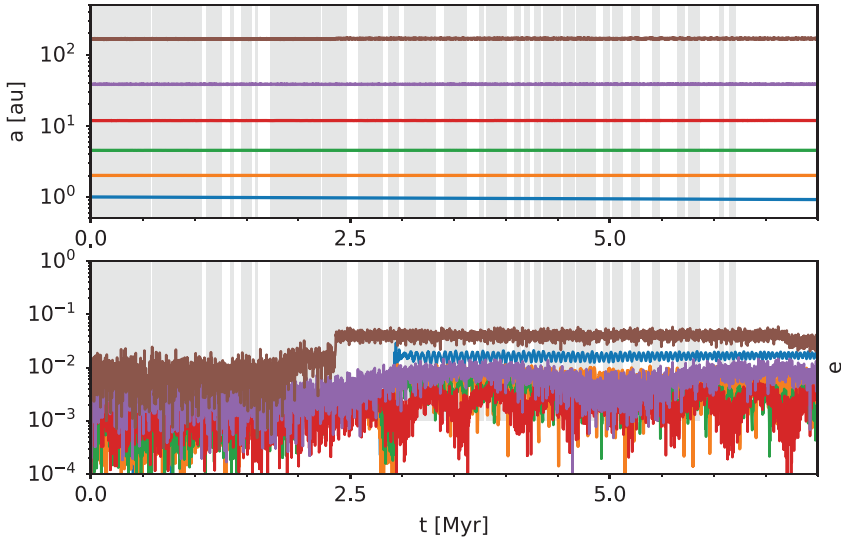


Figure 8.15. Second stage calculation for `LonelyPlanets`.

rapidly increase with time, and only four planets survive until an age of 3 Myr, when we stop the calculation.

It is relatively straightforward to include other effects in to the `LonelyPlanets` scripts, such as test particles (representing asteroids or comets), stellar evolution (representing not only the cluster stars but also the host stars of the planetary systems), etc. The method outlined here is a workable test version meant for integrating a single planetary system (composed of some 30 planets) in a cluster composed of 1000 stars among which is the planetary system.

Integrating the planetary system requires $\mathcal{O}[(1 + N_p + N_{cc})^2]$ operations per time step. Here, $f \leq 1$ indicates the fraction of time steps during which the planetary system is perturbed by the neighboring stars (the gray shaded areas in Figure 8.15). In the simplest case, $f \equiv 1$, indicating that the planetary system is always perturbed, irrespective of the mass or distance to the nearest perturber. This may be the case so long the planetary system is a cluster member, but as soon as the star escapes the cluster, $f \rightarrow 0$. This is nicely illustrated in Figure 8.15, where the last strong encounter with the planetary system, at an age of about 5.7 Myr, leads to a perturbation in the outermost planet (see the hiccup in the brown curve in Figure 8.15 at the same time) and causes the star to escape the cluster. The further integration of the planetary system is achieved by a standalone symplectic integrator. We can still include the galactic potential, in case we are interested in the evolution of the Oort cloud or asteroids along in the star’s galactic orbit.

8.8 Validation

Compared to other coupling strategies, problems with linear coupling are relatively easy to validate, because each of the individual parts can to some degree be validated separately, as described in previous chapters. Energy is generally not strictly

conserved in coupled problems, and neither is momentum, but wherever conservation laws do apply it is important to check them. Mass, number of objects, energy, and angular momentum are generally conserved in specific subproblems, but not throughout the simulation.

A simple validity check for conserving mass during stellar evolution and energy during stellar dynamics is

```
mass = stellar.particles.mass.sum()
stellar.evolve_model(time)
channel_stellar_to_framework.copy()
lost_mass = mass - stars.mass.sum()

channel_framework_to_gravity.copy()
Etot = gravity.kinetic_energy + gravity.potential_energy
gravity.evolve_model(time)
dE = Etot - (gravity.kinetic_energy + gravity.potential_energy)
```

Because mass is lost in stellar evolution, the kinetic and potential energies of the gravitational system are also affected. These can be checked independently, for example with

```
E_stellar = gravity.kinetic_energy + gravity.potential_energy
stellar.evolve_model(time)
channel_stellar_to_gravity.copy()
Etot = gravity.kinetic_energy + gravity.potential_energy
dE_stellar = E_stellar - Etot

gravity.evolve_model(time)
dE_gravity = Etot - (gravity.kinetic_energy + gravity.potential_energy)
```

Printing `dE_gravity` then provides a diagnostic on the quality of the simulation results. Similar strategies can be applied in hydrodynamics, and radiative transfer, as we will see in the next two chapters. We will experience similar issues in Chapter 9, and we postpone further discussion until there.

8.9 Assignments

8.9.1 Interlaced Timestepping

In Section 8.4.5, we discussed the evolution of the Wolf–Rayet colliding wind triple system θ Muscae, in which the WC5/6 star WR48 is in a 19 day orbit with an O6 companion, and both are orbited by a distant and relatively low-mass O9.5/B0Iab star. In the example presented in Section 8.4.5, stellar evolution and the dynamical evolution of the triple were applied at very short and constant time intervals of only one-tenth of the inner orbital period. It is not clear if such a small time step is actually needed to recover the correct physics. In this assignment we will study this.

In Section 8.4.4, we discussed some alternative time-stepping strategies, but there are other ways to do this. In this assignment, implement an alternative time-stepping scheme by allowing the time step to depend on the rate of mass loss, not allowing the total mass loss during the step to exceed a given value. If the time step becomes too large, the integrator will produce incorrect results, but the integration will take too

long for time steps that are too small. Use the interlaced temporal discretization method described in Section 8.4.4 to study if this is really a desirable way of integrating two numerical solvers.

Run a series of simulations, systematically halving the time step from one to the next, and record the semimajor axis and eccentricity of the inner and the outer binary and their relative inclination. Make a plot of the orbital parameters and the wall-clock computer time as a function of the time step size. Based on these curves, can you decide what is the best time step size for simulation of a triple system such as θ Muscae?

- How does the optimum time step depend on the mass-loss rate in the star with the strongest wind?
- How does optimum time step depend on the initial relative inclination between the inner and the outer orbit?

8.9.2 Stellar Evolution and Dynamics

Generate a Plummer sphere of $N = 1000$ stars with a Salpeter stellar mass function between $M_{\min} = 0.1 M_{\odot}$ and $M_{\max} = 100 M_{\odot}$ (power-law slope $\alpha = -2.3$). Evolve the system using an N -body integrator of your choice, including stellar evolution, and measure the moment when the cluster has lost half of its initial mass. Take, for example, the simple N -body code in Code example 4.1 as a starting point, and expand it with the stellar-evolution recipes from Code example 8.1.

Now change the power-law slope from $\alpha = -2.3$ (Salpeter) to $\alpha = -2.0$ and repeat the measurement. Continue this process with increasing slope until the initial mass function is flat. What can you conclude when you plot the time for the cluster to lose half of its initial mass as a function of the slope of the initial mass function?

- Why do clusters with relatively flat mass functions dissolve more quickly than those with a steep mass function?
- What changes when stellar evolution is switched off?
- Why is it not possible to continue the simulation to complete dissolution with a Salpeter mass function without stellar evolution?

8.9.3 Multiple Stellar Populations

Globular clusters had long been thought to contain single stellar populations—all stars formed at the same time with the same metallicity—but recent discoveries challenge this view. In particular, the discovery of numerous globular clusters, e.g., ω Centauri (Lee et al. 1999) and NGC2808 (Piotto et al. 2007), with two or more distinct main-sequence populations has been puzzling astronomers for at least two decades. One question that keeps cropping up is the rate at which two (or more) distinct stellar populations mix dynamically when their birth distributions differ. In this experiment, we will study this phenomenon by initializing a star cluster with two distinct stellar populations. These might be two populations with different metallicities, or with different ages.

Generate a first population of 500 stars by selecting masses from a Salpeter (or any other) mass distribution. Evolve the population to an age of 1 Gyr. Select

positions and velocities for these stars from a Plummer model with a virial radius of 1 pc, and scale the star cluster to virial equilibrium. Now generate a second stellar population at zero age and using the same (Salpeter) initial mass function, but now using a virialized Plummer sphere with a virial radius of 3 pc for the stars. Before starting the N -body integrator, you will need to merge the two stellar distributions and scale the whole system to virial equilibrium.

Evolving the two populations dynamically can be done in the same N -body integrator. Use two stellar-evolution codes for the two populations. Evolve the entire cluster for 1 Gyr and study how the individual Lagrangian radii of the two populations, and of the cluster as a whole, evolve with time from the start of the N -body integration.

References

- Aarseth, S. A. 2003, *Gravitational N-body Simulations* (Cambridge: Cambridge Univ. Press)
- Breslau, A., Steinhausen, M., Vincke, K., & Pfalzner, S. 2014, [A&A](#), **565**, [A130](#)
- Carathéodory, C. 1909, *MatAn*, **67**, 355
- Duquennoy, A., & Mayor, M. 1991, [A&A](#), **248**, [485](#)
- Eggleton, P. 2006, *Evolutionary Processes in Binary and Multiple Stars* (Cambridge: Cambridge Univ. Press)
- Eisenstein, D. J., & Hut, P. 1998, [ApJ](#), **498**, [137](#)
- Gaburov, E., Lombardi, J. C., & Portegies Zwart, S. 2008, [MNRAS](#), **383**, [L5](#)
- Goodwin, S. P., & Whitworth, A. P. 2004, [A&A](#), **413**, [929](#)
- Hill, G. M., Moffat, A. F. J., & St-Louis, N. 2002, [MNRAS](#), **335**, [1069](#)
- Hut, P. 1993, [ApJ](#), **403**, [256](#)
- Hut, P., Makino, J., & McMillan, S. 1995, [ApJL](#), **443**, [L93](#)
- Jänes, J., Pelupessy, I., & Portegies Zwart, S. 2014, [A&A](#), **570**, [A20](#)
- King, I. R. 1966, [AJ](#), **71**, [64](#)
- Kolmogorov, A. N. 1933, *Giornale della Istituto Italiano degli Attuari*, **4**, 83
- Kozai, Y. 1962, [AJ](#), **67**, [591](#)
- Kroupa, P. 2001, [MNRAS](#), **322**, [231](#)
- Lee, Y. W., Joo, J. M., Sohn, Y. J., et al. 1999, [Natur](#), **402**, [55](#)
- Lidov, M. L. 1962, [Planet. Space Sci.](#), **9**, [719](#)
- Lombardi, J., Rasio, C., & Shapiro, S. L., F. A. 1995, [ApJL](#), **445**, [L117](#)
- Lombardi, J. C., Thrall, A. P., Deneva, J. S., Fleming, S. W., & Grabowski, P. E. 2003, [MNRAS](#), **345**, [762](#)
- Lombardi, J., & Sawyer Warren, J. 2014, *MMAS: Make Me A Star*, [ascl:1412.010](#)
- Lombardi, J. C., Rasio, F. A., & Shapiro, S. L. 1996, [ApJ](#), **468**, [797](#)
- Lombardi, J. C., Warren, J. S., Rasio, F. A., Sills, A., & Warren, A. R. 2002, [ApJ](#), **568**, [939](#)
- MacPherson, G. J., Davis, A. M., & Zinner, E. K. 1995, [Metic](#), **30**, [365](#)
- Mardling, R. A. 2008, *IAU Symp. 246, Dynamical Evolution of Dense Stellar Systems*, ed. E. Vesperini, M. Giersz, & S. Sills (Cambridge: Cambridge Univ. Press) 246
- McMillan, S. L. W., & Hut, P. 1996, [ApJ](#), **467**, [348](#)
- Mikkola, S., & Merritt, D. 2008, [AJ](#), **135**, [2398](#)
- Piotto, G., Bedin, L. R., Anderson, J., et al. 2007, [ApJL](#), **661**, [L53](#)
- Plummer, H. C. 1911, [MNRAS](#), **71**, [460](#)
- Portegies Zwart, S. F., & van den Heuvel, E. P. J. 2016, [MNRAS](#), **456**, [3401](#)

- Portegies Zwart, S. F., & Verbunt, F. 1996, *A&A*, [309](#), [179](#)
- Portegies Zwart, S. F., & Verbunt, F. 2012, *SeBa: Stellar and Binary Evolution, Astrophysics Source Code Library*, [ascl:1201.003](#)
- Portegies Zwart, S. F., & Yungelson, L. R. 1998, *A&A*, [332](#), [173](#)
- Portegies Zwart, S. F., Makino, J., McMillan, S. L. W., & Hut, P. 1999, *A&A*, [348](#), [117](#)
- Salpeter, E. E. 1955, *ApJ*, [121](#), [161](#)
- Sanders, N. E., Soderberg, A. M., Gezari, S., et al. 2015, *ApJ*, [799](#), [208](#)
- Sills, A., & Lombardi, J. C. 1997, *ApJL*, [484](#), [L51](#)
- Smirnov, N. N. 1948, *Ann. Math. Stat.*, [19](#), [279](#)
- Sugawara, Y., Tsuboi, Y., & Maeda, Y. 2008, *A&A*, [490](#), [259](#)
- Tanikawa, A., & Fukushima, T. 2009, *PASJ*, [61](#), [721](#)
- Terlevich, E. 1987, *MNRAS*, [224](#), [193](#)
- Toonen, S., Nelemans, G., & Portegies Zwart, S. 2012, *A&A*, [546](#), [A70](#)
- van der Helm, M. I., Saladino, T. P. G., Pelupessy, F. I., Portegies Zwart, S., & Pols, O. R. 2019, *A&A*, [625](#), [85](#)
- Vicente, S. M., & Alves, J. 2005, *A&A*, [441](#), [195](#)
- von Zeipel, H. 1910, *AN*, [183](#), [345](#)
- White, S. D. M. 1994, arXiv:[astro-ph/9410043](#)